Lista de Contenido

- Lista de Contenido
- Temas para revisar para FullStack
  - Areas a revisar
    - 1. C# / .NET
    - 2. Angular
    - 3. Full Stack
    - 4. Preguntas de entrevista
  - Preguntas en general
  - Una estructura que funciona muy bien
    - Así puedes comparar objetivamente a varios candidatos.
- Donde hacer lo examenes
  - Las Opciones son:
    - TestGorilla
    - HackerRank
    - DevSkiller
    - TestDome
    - CodeSubmit
    - Intervue
  - Lo que yo usaría para tu caso
- Buscar opciones gratuitas
  - Para exámenes gratuitos
    - 1. W3Schools — Angular
    - 2. W3Schools — C#
    - 3. WeLoveDevs — Angular
    - 4. TixPAPA — C# Quiz
  - Plataformas con opción gratuita, pero ojo
    - TestGorilla
    - Adaface
  - Pero encontré algo MUY interesante para tu caso
  - Mi recomendación si no quieres pagar nada
    - Assessment Full Stack C# + Angular — 100 puntos
- Assessment Full Stack — C#/.NET + Angular
  - Estructura general
    - Criterio sugerido
- NIVEL JUNIOR
  - Parte 1 — C# — 20 puntos
    - Pregunta 1
    - Pregunta 2
    - Pregunta 3
    - Pregunta 4
    - Pregunta 5
  - Parte 2 — ASP.NET Core — 15 puntos

        - Pregunta 6
        - Pregunta 7
        - Pregunta 8
        - Pregunta 9
        - Pregunta 10
    - Parte 3 — Angular — 25 puntos
        - Pregunta 11
        - Pregunta 12
        - Pregunta 13
        - Pregunta 14
        - Pregunta 15
        - Pregunta 16
        - Pregunta 17
    - Parte 4 — SQL — 15 puntos
        - Pregunta 18
        - Pregunta 19
        - Pregunta 20
    - Parte 5 — Coding — 25 puntos
        - Ejercicio Junior
        - Bonus
- NIVEL MID
    - Parte 1 — C# avanzado
        - Pregunta 1
        - Pregunta 2
        - Pregunta 3
        - Pregunta 4
        - Pregunta 5
    - Parte 2 — ASP.NET Core
    - Parte 3 — Angular
        - RxJS
        - Forms
        - HTTP
        - Performance
    - Parte 4 — Coding Mid
        - Product Management
        - Requisitos adicionales
- NIVEL SENIOR
    - Parte 1 — Architecture
        - Caso
    - Senior — Code Review
    - Senior — Angular Code Review
    - Senior — Coding Challenge
    - Order Processing System
        - Order
        - Requerimientos
        - El candidato debe explicar

  - Rúbrica Senior
    - Interpretación
  - Lo que yo haría para tu proceso
- Definicion de Assessment Técnico
- Assessment Técnico — Full Stack Junior
  - Instrucciones para el candidato
  - Parte 1 — C# Fundamentals
    - 1. ¿Cuál es la diferencia principal entre una `class` y un `struct` en C#?
    - 2. ¿Qué imprime el siguiente código?
    - 3. ¿Cuál es el propósito de una interfaz?
    - 4. ¿Cuál es una ventaja de utilizar `async`/`await`?
    - 5. ¿Qué devuelve este método?
    - 6. ¿Qué hace LINQ?
    - 7. ¿Cuál es el resultado?
    - 8. ¿Para qué se utiliza `try/catch`?
    - 9. ¿Qué problema puede existir en este código?
    - 10. ¿Qué principio de programación recomienda que una clase tenga una única responsabilidad?
  - Parte 2 — ASP.NET Core / Web API
    - 11. ¿Qué es ASP.NET Core?
    - 12. ¿Qué HTTP status code representa normalmente una respuesta exitosa?
    - 13. ¿Qué status code utilizarías normalmente cuando un recurso no existe?
    - 14. ¿Qué método HTTP utilizarías normalmente para crear un nuevo recurso?
    - 15. ¿Qué es Dependency Injection?
    - 16. ¿Cuál es una responsabilidad típica de un Controller?
    - 17. ¿Qué es middleware en ASP.NET Core?
  - Parte 3 — Entity Framework Core / SQL
    - 18. ¿Qué es Entity Framework Core?
    - 19. ¿Qué representa normalmente un `DbContext`?
    - 20. ¿Qué hace `SaveChangesAsync()`?
    - 21. ¿Qué hace un `INNER JOIN`?
    - 22. ¿Qué hace `ORDER BY`?
    - 23. ¿Qué hace `GROUP BY`?
    - 24. Considera estas tablas:
  - Parte 4 — Angular / TypeScript
    - 25. ¿Qué es Angular?
    - 26. ¿Qué lenguaje se utiliza principalmente para desarrollar aplicaciones Angular?
    - 27. ¿Qué decorator se utiliza para definir un componente Angular?
    - 28. ¿Para qué se utiliza un Angular Service?
    - 29. ¿Qué herramienta se utiliza normalmente para realizar

HTTP requests en Angular?
    * 30. ¿Qué es un Observable?
    * 31. ¿Qué hace Angular Routing?
    * 32. ¿Para qué se utilizan los Reactive Forms?
    * 33. ¿Qué es TypeScript?
    * 34. ¿Qué hace `ngFor`?
  - Parte 5 — Full Stack / Integración
    * 35. Tienes:
    * 36. ¿Por qué no deberías conectar Angular directamente a SQL Server?
    * 37. ¿Qué formato se utiliza comúnmente para intercambiar datos entre Angular y ASP.NET Core?
    * 38. Un endpoint devuelve:
    * 39. Un endpoint devuelve:
  - Parte 6 — Preguntas de razonamiento
    * 40.
    * 41.
    * 42.
    * 43.
    * 44.
  - Parte 7 — Coding Challenge
  - Product API
    * Modelo
    * Endpoints requeridos
    * Requisitos
    * Bonus
    * Clave de respuestas
  - Guía de evaluación — Preguntas abiertas
    * Pregunta 40
    * Pregunta 41
    * Pregunta 42
    * Pregunta 43
    * Pregunta 44
    * Evaluación del Coding Challenge
      · Señales positivas
      · Red flags
    * Resultado final
      · 90–100 — Junior fuerte
      · 80–89 — Junior
      · 70–79 — Junior con áreas de mejora
      · 60–69 — Junior débil
      · <60 — No recomendado
    * Recomendación de interpretación
- Assessment Técnico — Full Stack Mid-Level
  - Assessment Técnico — Full Stack Mid-Level
  - Objetivo

- Instrucciones para el candidato
- PARTE 1 — C# / .NET
  - 1. LINQ y ejecución diferida
  - 2. `IEnumerable` vs `IQueryable`
  - 3. ¿Qué problema existe aquí?
- 4. Dependency Injection
  - 5. Lifetimes
  - 6. ¿Qué problema puede producir un `DbContext` Singleton?
  - 7. ¿Cuál es la diferencia principal entre `Task` y `Thread`?
  - 8. ¿Qué ventaja tiene `CancellationToken`?
  - 9. ¿Qué problema tiene este código?
  - 10. ¿Qué principio SOLID representa mejor la idea de depender de abstracciones?
- PARTE 2 — ASP.NET Core / Web API
  - 11. ¿Qué diferencia existe entre Authentication y Authorization?
  - 12. Un endpoint requiere autenticación y el usuario no proporciona credenciales válidas.
  - 13. Un usuario está autenticado pero no tiene permisos para ejecutar una operación.
  - 14. ¿Qué es un DTO?
  - 15. ¿Por qué evitarías devolver directamente una entidad EF Core desde un Controller?
  - 16. ¿Qué debería hacer una API ante un error inesperado?
  - 17. ¿Qué ventaja ofrece un Global Exception Handler?
  - 18. ¿Cuál sería una buena práctica para almacenar secretos como passwords de DB o API keys?
- PARTE 3 — Entity Framework Core / SQL
  - 19. ¿Qué problema puede existir aquí?
  - 20. ¿Cómo mejorarías una consulta de solo lectura?
  - 21. ¿Qué hace `Include()`?
  - 22. ¿Cuál consulta es preferible si solamente necesitas Id y Name?
  - 23. ¿Qué problema puede provocar traer millones de registros con `ToListAsync()`?
  - 24. ¿Qué solución utilizarías para una API que muestra 50 registros por página?
  - 25. ¿Qué hace un índice en una base de datos?
  - 26. ¿Qué riesgo existe en construir SQL concatenando directamente input del usuario?
- PARTE 4 — Angular / TypeScript
  - 27. ¿Cuál es una ventaja de utilizar Reactive Forms?
  - 28. ¿Qué problema existe aquí?
  - 29. ¿Qué problema es más evidente aquí?
  - 30. Si necesitas ejecutar tres requests independientes y esperar a que todas terminen, ¿qué operador podría ser apropiado?
  - 31. ¿Qué operador utilizarías frecuentemente para cancelar la

request anterior cuando llega un nuevo valor?
    * 32. ¿Qué escenario es apropiado para `debounceTime()`?
    * 33. ¿Qué hace un HTTP interceptor?
    * 34. ¿Para qué podría utilizarse un interceptor?
    * 35. ¿Qué estrategia puede ayudar a mejorar performance en Angular?
    * 36. ¿Qué problema puede existir al utilizar múltiples subscriptions manuales?
- PARTE 5 — Full Stack / API Integration
    * 37. Un usuario hace login correctamente.
    * 38. ¿Dónde debería validarse una regla de negocio crítica como:
    * 39. Angular recibe:
    * 40. ¿Por qué una API debería implementar pagination para grandes cantidades de datos?
    * 41. Un endpoint funciona en Postman pero falla desde Angular con CORS.
- PARTE 6 — Preguntas abiertas / troubleshooting
    * 42. Performance de API
    * 43. Angular
    * 44. Database
    * 45. Authentication
    * 46. Production debugging
- PARTE 7 — Coding Challenge #1
- Product Management API
    * Entidad
    * Requerimientos
    * Requisitos técnicos
    * Bonus
- PARTE 8 — Coding Challenge #2
    * Angular Product Screen
    * Requisitos
    * Bonus
    * CLAVE DE RESPUESTAS
- GUÍA DE EVALUACIÓN DE PREGUNTAS ABIERTAS
    * Pregunta 42 — API Performance
    * Pregunta 43 — Requests duplicadas
    * Pregunta 44 — Query lenta
    * Pregunta 45 — JWT / 401
    * Pregunta 46 — Registro duplicado
        · Frontend
        · Backend
    * RÚBRICA DEL CODING CHALLENGE #1
        · Red flags
    * RÚBRICA DEL CODING CHALLENGE #2
    * SCORING FINAL
    * INTERPRETACIÓN

            - 90–100 — Mid-Level fuerte
            - 80–89 — Mid-Level
            - 70–79 — Mid-Level con áreas de mejora
            - 60–69 — Junior avanzado
            - <60 — Junior / No recomendado
            - MÍNIMOS RECOMENDADOS
            - INDICADORES DE UN VERDADERO MID-LEVEL
            - Pregunta final recomendada
- Assessment Técnico — Full Stack Senior
  - Assessment Técnico — Full Stack Senior
  - Objetivo
    - INSTRUCCIONES PARA EL CANDIDATO
  - PARTE 1 — C# / .NET AVANZADO
    - 1. Async / Await
    - 2. Dependency Injection
    - 3. Garbage Collection
    - 4. `IDisposable`
    - 5. Records
    - 6. Concurrencia
    - 7. ¿Cuál es una estrategia posible para resolver el problema anterior?
    - 8. SOLID
  - PARTE 2 — ASP.NET CORE / API DESIGN
    - 9. Global Exception Handling
    - 10. API Versioning
    - 11. Idempotency
    - 12. Rate Limiting
    - 13. Authentication vs Authorization
    - 14. ¿Qué problema existe al confiar exclusivamente en autorización del frontend?
    - 15. ¿Cuál es una buena práctica para API secrets?
    - 16. ¿Qué información no debería devolverse normalmente en una respuesta de error de producción?
  - PARTE 3 — EF CORE / SQL / PERFORMANCE
    - 17. ¿Qué problema existe?
    - 18. ¿Qué alternativa podría ser apropiada si solo necesitas información específica?
    - 19. ¿Qué problema puede producir `Skip()` con offsets extremadamente grandes?
    - 20. ¿Cuándo considerarías keyset/cursor pagination?
    - 21. ¿Qué herramienta utilizarías para investigar por qué una query SQL está lenta?
    - 22. ¿Qué problema puede producir un índice excesivo?
    - 23. ¿Qué es un deadlock?
    - 24. ¿Qué utilizarías para investigar bloqueos o deadlocks?
  - PARTE 4 — ANGULAR / RXJS AVANZADO

    - 25. ¿Qué problema existe?
    - 26. ¿Qué combinación es apropiada para un search box?
    - 27. ¿Por qué `switchMap` puede ser apropiado para búsquedas?
    - 28. ¿Cuándo usarías `forkJoin`?
    - 29. ¿Qué ventaja puede proporcionar `ChangeDetectionStrategy.OnPush`?
    - 30. ¿Cuál es una estrategia adecuada para listas muy grandes?
    - 31. ¿Qué problema puede causar mantener subscriptions largas sin control?
    - 32. ¿Qué debería considerar un Senior al diseñar un Angular Service?
- PARTE 5 — SECURITY
    - 33. ¿Dónde debería validarse autorización para una operación crítica?
    - 34. ¿Qué es SQL Injection?
    - 35. ¿Cómo reducirías riesgo de SQL Injection?
    - 36. ¿Qué problema tiene guardar información sensible en `localStorage`?
    - 37. ¿Qué es XSS?
    - 38. ¿Qué estrategia ayuda contra XSS?
- PARTE 6 — SYSTEM DESIGN
    - 39. Diseño de sistema
    - A. Arquitectura
    - B. Database
    - C. Caching
    - D. Messaging
    - E. Reliability
    - F. Observability
    - G. Security
    - H. Scalability
- PARTE 7 — PRODUCTION TROUBLESHOOTING
    - 40. API lenta
    - 41. Memory leak
    - 42. Angular lento
    - 43. Requests duplicadas
    - 44. Error intermitente
- PARTE 8 — CODE REVIEW BACKEND
    - 45. Identifica los problemas
- PARTE 9 — CODE REVIEW ANGULAR
    - 46. Identifica problemas y propone mejoras.
- PARTE 10 — CODING CHALLENGE
- Order Processing API
    - Modelo
    - Requisitos
    - Consideraciones
- PARTE 11 — CODING CHALLENGE ANGULAR
    - Requisitos

        - CLAVE DE RESPUESTAS
        - RÚBRICA SYSTEM DESIGN
        - Excelente — 9–10 puntos
        - Bueno — 7–8 puntos
        - Regular — 5–6 puntos
        - Débil — <5 puntos
        - RÚBRICA CODE REVIEW BACKEND
        - RÚBRICA CODE REVIEW ANGULAR
        - RÚBRICA CODING BACKEND
        - RÚBRICA CODING ANGULAR
        - SCORING FINAL
        - INTERPRETACIÓN DEL RESULTADO
            - 90–100 — Senior fuerte
            - 80–89 — Senior
            - 70–79 — Mid/Senior
            - 60–69 — Mid-Level fuerte
            - <60 — No Senior
        - MÍNIMOS RECOMENDADOS
        - RED FLAGS DE SENIOR
            - 1. “Lo arreglo en Angular”
            - 2. “Más microservices”
            - 3. “Le pongo un índice”
            - 4. “Uso Singleton para mejorar performance”
            - 5. “Hago retry”
            - 6. “Deshabilito el botón”
            - 7. “Manejo todo con try/catch”
            - 8. No puede explicar su propio código
    - PREGUNTAS FINALES PARA LA ENTREVISTA
        - Pregunta 1
        - Pregunta 2
        - Pregunta 3
        - Pregunta 4
        - Pregunta 5
        - DECISIÓN FINAL
- Prueba de Código Full Stack — Mid-Level
    - Prueba de Código Full Stack — Mid-Level
    - C#/.NET + Angular + SQL Server
        - 1. Objetivo
        - 2. Escenario
        - 3. Modelo de datos
            - Product
            - Category
            - Order
            - OrderItem
        - 4. Backend
        - 5. Product API

    - Obtener productos
    - Obtener producto
    - Crear producto
    - Actualizar producto
    - Eliminar producto
    - 6. Category API
    - 7. Order API
    - 8. Consistencia
    - 9. Consultar órdenes
    - 10. Obtener detalle de una orden
    - 11. Requisitos de Backend
    - 12. Angular
    - 13. Product List
    - 14. Product Form
    - 15. Edit Product
    - 16. Orders
    - 17. Create Order
    - 18. RxJS
    - 19. Angular Architecture
    - 20. Manejo de errores
    - 21. Requisitos adicionales
        - Backend
        - Angular
    - 22. Testing
        - Backend
        - Angular
    - 23. Bonus
    - 24. README
    - Requirements
    - Installation
    - Database
    - Configuration
    - Architecture
    - Decisions
    - 25. Entregables
    - 26. Criterios de evaluación
    - 27. Evaluación del nivel
        - 90–100
        - 80–89
        - 70–79
        - 60–69
        - <60
    - 28. Restricciones
    - 29. Pregunta final
    - Mi recomendación
- Prueba de Código Full Stack — Senior

- C#/.NET + Angular + SQL Server
- 1. Objetivo
- 2. Escenario
- 3. Modelo de dominio
  - Product
  - Order
  - OrderItem
  - Customer
- 4. Regla principal de negocio
- 5. Backend
- 6. Product API
- 7. Product Detail
- 8. Create Product
- 9. Update Product
- 10. Order API
- 11. Concurrencia de inventario
- 12. Idempotencia
- 13. Transaction Boundary
- 14. External Service
- 15. Consultar órdenes
- 16. Order Detail
- 17. Cancel Order
- 18. Security
- 19. API Error Handling
- 20. SQL / EF Core
  - N+1 queries
  - Over-fetching
  - Pagination en memoria
- 21. Performance
- 22. Angular
- 23. Product Screen
- 24. Create Order
- 25. RxJS
- 26. Manejo de estado
- 27. HTTP Interceptor
- 28. Performance Angular
- 29. Testing
  - Backend
  - Angular
- 30. Observabilidad
- 31. Health Check
- 32. Docker
- 33. Documentación
  - Setup
  - Architecture
  - Database

    - API
    - Decisions
    - Concurrency
    - Idempotency
    - External Services
    - Improvements
- 34. Parte arquitectónica obligatoria
    - Escenario A
    - Escenario B
    - Escenario C
    - Escenario D
- 35. Restricciones
- 36. Entregables
- 37. Criterios de evaluación
- 38. Bonus
    - +2
    - +2
    - +1
    - +1
    - +1
    - +1
    - +1
    - +1
    - +1
- 39. Rúbrica de Seniority
    - 90–100
        - Senior fuerte
    - 80–89
        - Senior
    - 70–79
        - Mid/Senior
    - 60–69
        - Mid-Level fuerte
    - <60
        - No alcanza Senior
- 40. Red Flags
    - Red Flag 1
    - Red Flag 2
    - Red Flag 3
    - Red Flag 4
    - Red Flag 5
    - Red Flag 6
    - Red Flag 7
    - Red Flag 8
    - Red Flag 9
    - Red Flag 10

- 41. Preguntas de revisión durante la entrevista
  - Pregunta 1
  - Pregunta 2
  - Pregunta 3
  - Pregunta 4
  - Pregunta 5
  - Pregunta 6
  - Pregunta 7
  - Pregunta 8
  - Pregunta 9
  - Pregunta 10
- 42. Decisión final
  - Una diferencia importante respecto al Mid

Temas para revisar para FullStack

> requiero hacer assesments para full stack con C# y angular sabes como puedo hacerlo?

Sí. Si necesitas crear **assessments técnicos para Full Stack .NET/C# + Angular**, puedo ayudarte a diseñarlos desde cero.

Areas a revisar

Te recomiendo que el assessment evalúe **4 áreas**:

1. C# / .NET

- C# moderno: LINQ, async/await, generics, interfaces
- OOP y SOLID
- Dependency Injection
- ASP.NET Core Web API
- Entity Framework Core
- Manejo de errores y validaciones
- Authentication/Authorization con JWT
- Unit testing

2. Angular

- Components
- Services e Injection
- RxJS / Observables

- Routing
- Reactive Forms
- HTTP Client
- Guards / interceptors
- Manejo de estado
- Performance
- Unit testing

3. Full Stack

Aquí es donde puedes diferenciar un candidato realmente bueno.

Por ejemplo, pedirle construir:

Product Management API + Angular UI

El candidato tendría que:

- Crear una API REST en ASP.NET Core.
- Usar EF Core con SQL Server.
- Implementar CRUD de productos.
- Agregar búsqueda, filtros y paginación.
- Implementar autenticación JWT.
- Crear la aplicación Angular.
- Consumir la API mediante un service.
- Crear formulario de alta/edición.
- Manejar errores y loading states.
- Escribir tests.
- Documentar cómo ejecutar el proyecto.

4. Preguntas de entrevista

Preguntas en general

Además del coding assessment, puedes tener preguntas como:

C#

- ¿Cuál es la diferencia entre `IEnumerable`, `IQueryable` y `List`?
- ¿Qué problema resuelve Dependency Injection?
- ¿Cuándo utilizarías `async/await`?
- ¿Qué es `CancellationToken`?
- ¿Cómo evitarías un N+1 query con EF Core?

Angular

- ¿Cuál es la diferencia entre Observable y Promise?
- ¿Cuándo usarías `Subject` o `BehaviorSubject`?

- ¿Qué hace un HTTP interceptor?
- ¿Cómo manejarías un formulario complejo?
- ¿Cómo optimizarías una aplicación Angular con muchos componentes?

Architecture

- ¿Cómo estructurarías una aplicación ASP.NET Core + Angular?
- ¿Dónde colocarías las validaciones?
- ¿Cómo manejarías errores entre API y frontend?
- ¿Cómo implementarías autorización por roles?
- ¿Qué harías para evitar que un usuario modifique recursos que no le pertenecen?

Una estructura que funciona muy bien

Yo haría **3 niveles**:

Nivel	Duración	Objetivo
Junior	60–90 min	Fundamentos C# + Angular
Mid-level	2–3 h	API + Angular + DB
Senior	3–5 h	Full stack + arquitectura + testing

Y no evaluaría solamente si "funciona". Le pondría una rúbrica, por ejemplo:

- **C# / .NET:** 25%
- **Angular:** 20%
- **Database / EF Core:** 15%
- **Architecture:** 15%
- **Testing:** 10%
- **Code quality:** 10%
- **Documentation:** 5%

Así puedes comparar objetivamente a varios candidatos.

Si quieres, puedo prepararte **un assessment completo listo para enviar a candidatos**, incluyendo **el enunciado, repositorio inicial sugerido, requisitos funcionales, bonus points, preguntas de entrevista y una rúbrica de evaluación de 100 puntos**, específicamente para **C#/.NET + Angular**.

Donde hacer lo examenes

Lugares para hacer examenes en linea...

Las Opciones son:

Sí. Hay varias plataformas donde puedes **tomar assessments ya preparados** o incluso **crear uno propio para candidatos**. Para Full Stack **C#/.NET + Angular**, estas son las que más te recomendaría:

TestGorilla

TestGorilla – .NET Assessment

TestGorilla – Angular Assessment

Tiene tests separados de **.NET y Angular**, con preguntas ya preparadas. Puedes combinarlos en un assessment y obtener resultados comparables entre candidatos.

Me gusta para un proceso de selección porque es relativamente sencillo de configurar.

HackerRank

HackerRank

Es muy bueno si quieres evaluar **programación real**, no solamente preguntas de opción múltiple.

Puedes evaluar:

- C#
- .NET
- SQL
- Angular/frontend
- Algoritmos
- Coding challenges

Incluso puedes crear assessments donde el candidato tiene que modificar código existente. Hay ejemplos públicos de assessments de Angular y C#/.NET. HackerRank+1

DevSkiller

DevSkiller – .NET Coding Tests

Esta me parece **muy interesante para un Full Stack Mid/Senior**.

En lugar de preguntarle simplemente:

> "¿Qué es Dependency Injection?"

puedes darle un proyecto .NET existente y pedirle que implemente funcionalidades, corrija bugs o mejore el código. DevSkiller puede ejecutar tests automáticamente y evaluar el resultado. GitHub

TestDome

TestDome – Angular Online Test

Tiene un assessment específico de Angular basado en **coding problems reales**, no solamente MCQ. También tiene pruebas para otras tecnologías.

CodeSubmit

CodeSubmit – Angular Coding Assignments

Es interesante si quieres hacer un **take-home assessment**. Tiene challenges de Angular que el candidato puede resolver usando su propio IDE y luego tú puedes revisar el código.

Intervue

Intervue – Developer Assessments

Tiene bancos de preguntas para **Angular, ASP.NET, SQL, JavaScript**, etc. Su sitio indica más de 1,000 preguntas en distintas tecnologías. Intervue

---

Lo que yo usaría para tu caso

Si estás buscando **contratar Full Stack C# + Angular**, no me quedaría únicamente con un examen de preguntas.

Haría algo así:

Assessment de 90–120 minutos

- 15 preguntas C#/.NET → **20 puntos**
- 10 preguntas Angular/TypeScript → **15 puntos**
- 5 preguntas SQL/EF Core → **10 puntos**
- 1 ejercicio práctico C#/.NET → **25 puntos**
- 1 ejercicio práctico Angular → **20 puntos**
- Code quality / architecture → **10 puntos**

  Total: 100 puntos

Y para un **Senior**, agregaría un ejercicio tipo:

> "Tienes esta API ASP.NET Core y esta aplicación Angular. Hay 5 problemas intencionales. Identifica los problemas, corrígelos y explica tus decisiones."

Eso te permite detectar muchísimo mejor la diferencia entre alguien que **realmente ha trabajado con .NET + Angular** y alguien que solamente memorizó respuestas.

Si me dices si buscas **Junior, Mid o Senior**, puedo buscarte **3–5 assessments concretos que puedas usar**, preferiblemente gratuitos o con trial, y decirte cuál escogería para tu proceso.

Sources

Buscar opciones gratuitas

> falto agregar, que sean gratuitos

Sí. Si el requisito es **que sean realmente gratuitos**, cambia bastante la recomendación. Revisé opciones actuales y separaría entre **practicar tú** y **usar el examen para evaluar candidatos**.

Para exámenes gratuitos

1. W3Schools — Angular

Angular Quiz de W3Schools

Tiene un **quiz gratuito de Angular** y ejercicios interactivos. Es bueno para evaluar fundamentos como componentes, directivas, servicios, routing, etc. W3Schools

Ideal: Junior / fundamentos.

---

2. W3Schools — C

C# Quiz de W3Schools

Tiene quizzes gratuitos de C# y ejercicios para practicar conceptos del lenguaje.

Ideal: Junior–Mid.

---

3. WeLoveDevs — Angular

Angular Fundamentals Test – WeLoveDevs

Esta me gusta bastante para lo que buscas. Tiene un examen de Angular con preguntas aleatorias; actualmente indica **20 preguntas seleccionadas de un banco de 46** y permite hacerlo online. WeLoveDevs

Ideal: Junior / Mid.

---

4. TixPAPA — C# Quiz

C# Quiz gratuito

Es un quiz gratuito de C# con temas como:

- Value vs reference types
- Properties
- LINQ
- Generics
- Exceptions

Además muestra explicación de las respuestas. TixPapa

Ideal: filtro rápido de C#.

Plataformas con opción gratuita, pero ojo

TestGorilla

TestGorilla – Angular Test

Tiene un test de Angular y permite **ver una demo/prueba**, pero no asumiría que puedes administrar un proceso completo de selección ilimitado gratis. Su test cubre componentes, directives, pipes, forms, services y RxJS. TestGorilla

Adaface

Adaface – Angular Online Test

Tiene un test bastante bueno: **40 minutos**, preguntas de Angular/JavaScript/HTML/CSS y una pregunta de coding. La página actualmente ofrece comenzar gratuitamente, pero algunas funciones de administración/candidatos pueden estar sujetas al plan. Adaface

---

Pero encontré algo MUY interesante para tu caso

Existe un assessment específicamente llamado:

Fullstack Developer (C# .NET + Angular)

Fullstack Developer C# .NET + Angular Assessment

Está diseñado exactamente alrededor de:

- .NET
- .NET MVC
- ASP.NET Core MVC
- Angular
- SQL Server
- Integración Angular + .NET
- Coding

Incluso tiene un ejercicio práctico de **construir un Online Bookstore usando .NET + Angular + SQL Server**. WeCP

Peeeero: aunque puedes revisar el contenido, no lo pondría en la categoría de "100% gratis para administrar candidatos" sin comprobar su plan actual.

Mi recomendación si no quieres pagar nada

Si tu objetivo es **evaluar candidatos y no simplemente estudiar**, yo haría algo diferente:

No dependería de una plataforma.

Puedes utilizar:

1. **W3Schools / WeLoveDevs** → preguntas de conocimiento.
2. **GitHub** → proyecto práctico.
3. **Google Forms / Microsoft Forms** → preguntas teóricas.
4. **GitHub Classroom** o un repositorio → ejercicio de programación.
5. Yo te puedo crear la **rúbrica automática/manual de evaluación**.

Por ejemplo:

Assessment Full Stack C# + Angular — 100 puntos

Parte A — C# / .NET — 25 pts

- OOP
- SOLID
- LINQ
- async/await
- Dependency Injection
- Web API
- EF Core

Parte B — Angular — 25 pts

- Components
- Services
- RxJS
- Reactive Forms

- Routing
- Guards
- HTTP Interceptors

Parte C — SQL — 15 pts

- JOINs
- GROUP BY
- índices
- relaciones
- queries

Parte D — Coding — 25 pts

Crear:

> **Product Management System**

Backend:
`ASP.NET Core Web API + EF Core + SQL Server`

Frontend:
`Angular`

Con:

- Login
- CRUD
- Search
- Pagination
- Validation
- Error handling

Parte E — Code Review — 10 pts

Darles código con errores intencionales y pedir:

> "Identify the problems and explain how you would improve them."

Esto último es **muy bueno para distinguir Senior de alguien que simplemente sabe responder preguntas de memoria**.

Si quieres, puedo armarte ahora mismo un **assessment completo de 50 preguntas + 2 ejercicios de código para Full Stack C# + Angular**, separado en **Junior / Mid / Senior**, y que puedas copiar directamente a **Google Forms** sin pagar ninguna plataforma.

Sources

Assessment Full Stack — C#/.NET + Angular

> Si arma el assesment distinguiendo los niveles…

Sí. Te propongo un assessment que puedas usar como **proceso real de selección**, no solamente como quiz. Lo dividiría en **Junior, Mid y Senior**, aumentando progresivamente la dificultad y el peso de arquitectura/código.

A continuación te dejo una versión completa que puedes pasar a Google Forms, Microsoft Forms o usar como guía para una entrevista técnica.

Estructura general

Nivel	Duración	Preguntas	Coding	Total
Junior	60–75 min	30	1 ejercicio	100 pts
Mid	90–120 min	35	2 ejercicios	100 pts
Senior	120–180 min	35	2 ejercicios + arquitectura	100 pts

Criterio sugerido

- **0–49:** No alcanza el nivel
- **50–64:** Débil / requiere supervisión
- **65–79:** Cumple
- **80–89:** Buen candidato
- **90–100:** Excelente

NIVEL JUNIOR

Parte 1 — C# — 20 puntos

Pregunta 1

¿Cuál es la diferencia entre `class` y `struct` en C#?

A. No existe diferencia
B. `class` es reference type y `struct` es value type
C. `struct` es reference type y `class` value type
D. Ambos son siempre reference types

Respuesta: B

Pregunta 2

¿Qué palabra clave se utiliza para implementar una interfaz?

A. `inherits`
B. `extends`
C. `implements`
D. `:`

Respuesta: D

Pregunta 3

¿Qué hace `async/await`?

A. Crea automáticamente nuevos threads
B. Permite trabajar con operaciones asíncronas de forma más sencilla
C. Hace que el código sea siempre más rápido
D. Reemplaza completamente los threads

Respuesta: B

Pregunta 4

¿Qué devuelve normalmente un método declarado como:

```
public async Task<User> GetUserAsync()
```

A. `User`
B. `Task`
C. `Task<User>`
D. `void`

Respuesta: C

Pregunta 5

¿Qué hace LINQ?

A. Conecta automáticamente con SQL Server
B. Permite consultar y transformar colecciones/datos
C. Reemplaza Entity Framework
D. Es un sistema de autenticación

Respuesta: B

Parte 2 — ASP.NET Core — 15 puntos

Pregunta 6

¿Qué es Dependency Injection?

A. Una técnica para almacenar datos
B. Un patrón que permite proporcionar dependencias a una clase desde fuera
C. Un sistema de autenticación
D. Un tipo de middleware

Respuesta: B

Pregunta 7

¿Cuál es una responsabilidad típica de un Controller?

A. Dibujar componentes Angular
B. Recibir HTTP requests y devolver responses
C. Ejecutar directamente JavaScript
D. Crear tablas SQL

Respuesta: B

Pregunta 8

¿Qué HTTP status code representa normalmente “Not Found”?

A. 200
B. 201
C. 400
D. 404

Respuesta: D

Pregunta 9

¿Qué status code corresponde normalmente a una creación exitosa?

A. 200
B. 201
C. 204
D. 500

Respuesta: B

Pregunta 10

¿Qué es middleware en ASP.NET Core?

A. Una tabla de SQL
B. Un componente que participa en el pipeline HTTP
C. Un componente Angular
D. Una clase Entity Framework

Respuesta: B

Parte 3 — Angular — 25 puntos

Pregunta 11

¿Qué es un Angular Component?

A. Una tabla SQL
B. Una unidad de UI con template, lógica y metadata
C. Un endpoint
D. Un servicio backend

Respuesta: B

Pregunta 12

¿Qué decorador identifica normalmente un componente?

A. `@Service`
B. `@Component`
C. `@Controller`
D. `@Inject`

Respuesta: B

Pregunta 13

¿Para qué se utiliza `HttpClient`?

A. Crear componentes
B. Hacer HTTP requests
C. Crear bases de datos
D. Compilar TypeScript

Respuesta: B

Pregunta 14

¿Qué es un Observable?

A. Un tipo de base de datos
B. Una representación de una secuencia de valores que puede observarse a lo largo del tiempo
C. Un componente Angular
D. Un endpoint REST

Respuesta: B

Pregunta 15

¿Cuál es la extensión habitual de un archivo TypeScript?

A. `.js`
B. `.ts`
C. `.cs`
D. `.tsx`

Respuesta: B

Pregunta 16

¿Para qué sirve Angular Routing?

A. Para acceder a SQL Server
B. Para manejar navegación entre vistas/componentes
C. Para crear APIs
D. Para compilar C#

Respuesta: B

Pregunta 17

¿Qué es un Service en Angular?

A. Una forma de encapsular lógica reutilizable/de acceso a datos
B. Un Controller de .NET
C. Una tabla
D. Un componente visual obligatorio

Respuesta: A

Parte 4 — SQL — 15 puntos

Pregunta 18

¿Qué hace un `INNER JOIN`?

A. Devuelve solamente registros con coincidencia entre las tablas
B. Devuelve todos los registros de ambas tablas
C. Elimina registros
D. Crea una tabla

Respuesta: A

Pregunta 19

¿Qué hace `GROUP BY`?

A. Ordena registros
B. Agrupa registros para realizar agregaciones
C. Elimina duplicados siempre
D. Crea índices

Respuesta: B

Pregunta 20

¿Cuál utilizarías para ordenar resultados?

A. `SORT`
B. `ORDER BY`
C. `GROUP`
D. `ARRANGE`

Respuesta: B

Parte 5 — Coding — 25 puntos

Ejercicio Junior

Crear una API ASP.NET Core que permita administrar productos.

Entidad:

```
Product
----------------
Id
Name
```

```
Price
Stock
```

Implementar:

```
GET    /api/products
GET    /api/products/{id}
POST   /api/products
PUT    /api/products/{id}
DELETE /api/products/{id}
```

Debe incluir:

- Entity
- Controller
- Service
- Repository o EF Core
- Validación básica
- Manejo básico de errores

Bonus

- Unit tests
- Swagger
- DTOs
- Async/await

NIVEL MID

Aquí ya no buscamos solamente que conozca la sintaxis. Queremos comprobar que **puede construir una aplicación profesional**.

Parte 1 — C# avanzado

Pregunta 1

¿Cuál es la diferencia entre:

`IEnumerable<T>`

y

`IQueryable<T>`

Respuesta esperada:

`IEnumerable` normalmente trabaja sobre datos ya materializados/en memoria, mientras que `IQueryable` permite construir una expresión que puede ser traducida por un proveedor, como EF Core, y ejecutarse en la fuente de datos.

Pregunta 2

¿Qué problema puede producir este código?

```
var users = context.Users.ToList();

var result = users
    .Where(x => x.IsActive)
    .ToList();
```

Respuesta esperada:

Está trayendo todos los usuarios a memoria antes de filtrar. Sería preferible filtrar en la consulta:

```
var result = await context.Users
    .Where(x => x.IsActive)
    .ToListAsync();
```

Pregunta 3

¿Qué es el problema N+1?

Respuesta esperada:

Cuando una consulta inicial obtiene N registros y posteriormente se ejecutan consultas adicionales individualmente para cada registro, generando potencialmente muchas llamadas a la base de datos.

Pregunta 4

¿Cuándo usarías `AsNoTracking()`?

Respuesta esperada:

Cuando solamente se necesitan leer entidades y no se planea modificarlas mediante el DbContext.

Pregunta 5

¿Qué ventajas tiene usar DTOs?

Debe mencionar varias de:

- Separación entre API y entidades

- Evitar exponer propiedades internas
- Controlar el contrato
- Validación
- Seguridad
- Evitar problemas de serialización/ciclos

Parte 2 — ASP.NET Core

Preguntas:

- ¿Cómo implementarías JWT authentication?
- Diferencia entre Authentication y Authorization.
- ¿Qué es un middleware?
- ¿Qué diferencia hay entre Singleton, Scoped y Transient?
- ¿Cómo manejarías excepciones globalmente?
- ¿Dónde colocarías validaciones?
- ¿Cómo versionarías una API?
- ¿Cómo manejarías pagination?
- ¿Cómo protegerías endpoints por roles?
- ¿Qué información nunca deberías poner en `appsettings.json`?

Parte 3 — Angular

Preguntas:

RxJS

Explica la diferencia entre:

```
Observable
Subject
BehaviorSubject
Promise
```

Forms

¿Cuándo utilizarías Reactive Forms?

HTTP

¿Cómo implementarías un interceptor para:

- agregar JWT
- manejar 401
- logging

- errores globales

Performance

¿Cómo mejorarías una aplicación Angular que tiene una tabla con 10,000 registros?

Esperaría respuestas como:

- Pagination
- Lazy loading
- OnPush
- trackBy
- Virtual scrolling
- Evitar subscriptions innecesarias
- Optimización de requests

Parte 4 — Coding Mid

Crear una aplicación:

Product Management

Backend:

```
ASP.NET Core
EF Core
SQL Server
JWT
```

Frontend:

```
Angular
TypeScript
RxJS
```

Funciones:

- Login
- List products
- Create
- Edit
- Delete
- Search
- Pagination
- Validation
- Error handling

Requisitos adicionales

El candidato debe:

- Utilizar DTOs.
- Utilizar Dependency Injection.
- Utilizar async/await.
- Implementar manejo de errores.
- Separar responsabilidades.
- Escribir al menos 3 unit tests.
- Utilizar Reactive Forms.
- Utilizar un HTTP interceptor.

NIVEL SENIOR

Aquí cambiaría completamente el enfoque.

Un Senior no debería aprobar simplemente porque respondió correctamente 30 preguntas.

Debe demostrar **criterio técnico y capacidad de tomar decisiones**.

Parte 1 — Architecture

Caso

Tenemos:

```
Angular
   |
   ↓
ASP.NET Core API
   |
   ↓
SQL Server
```

La aplicación tiene:

- 500,000 usuarios
- 5 millones de productos
- 100,000 requests/minuto
- múltiples roles
- auditoría
- integración con servicios externos

Preguntas:

1. ¿Cómo estructurarías el backend?
2. ¿Qué arquitectura utilizarías?

3. ¿Dónde pondrías la lógica de negocio?
4. ¿Cómo manejarías transacciones?
5. ¿Cómo implementarías caching?
6. ¿Cómo manejarías logging?
7. ¿Cómo implementarías observabilidad?
8. ¿Cómo manejarías errores?
9. ¿Cómo protegerías la API?
10. ¿Qué problemas esperas de SQL Server?

Aquí **no pondría respuestas múltiples**.

Evaluaría el razonamiento.

Senior — Code Review

Entrega al candidato este código:

```
[HttpGet]
public IActionResult GetUsers()
{
    var users = _context.Users.ToList();

    foreach (var user in users)
    {
        user.Orders = _context.Orders
            .Where(x => x.UserId == user.Id)
            .ToList();
    }

    return Ok(users);
}
```

Pregúntale:

> Identifica todos los problemas que encuentres y propón una solución.

Un Senior debería detectar cosas como:

- N+1 queries
- Materialización innecesaria
- Entidades directamente expuestas
- Falta de DTO
- Falta de async
- Posibles problemas de performance
- Falta de pagination
- Potenciales problemas de serialización
- Falta de cancellation
- Ausencia de manejo de errores

Senior — Angular Code Review

```
ngOnInit() {
  this.http.get('/api/products').subscribe(data => {
    this.products = data;

    this.http.get('/api/categories').subscribe(categories => {
      this.categories = categories;

      this.http.get('/api/suppliers').subscribe(suppliers => {
        this.suppliers = suppliers;
      });
    });
  });
}
```

Pregúntale:

> ¿Qué problemas tiene este código y cómo lo mejorarías?

Una respuesta fuerte debería considerar:

```
Nested subscriptions
        ↓
RxJS operators
        ↓
forkJoin / switchMap / combineLatest
        ↓
unsubscribe/lifecycle management
        ↓
loading/error states
```

Y debería poder explicar **por qué** elegiría cada alternativa.

Senior — Coding Challenge

Order Processing System

Crear un sistema para procesar órdenes.

Order

```
Order
    Id
    CustomerId
```

```
CreatedAt
Status
Items
      ProductId
      Quantity
      UnitPrice
```

Requerimientos

El sistema debe:

- Crear órdenes.
- Validar stock.
- Calcular total.
- Aplicar descuentos.
- Actualizar inventario.
- Persistir la orden.
- Evitar órdenes inconsistentes.
- Manejar concurrencia.
- Permitir consultar órdenes.
- Filtrar por status.
- Paginar resultados.

El candidato debe explicar

- Arquitectura
- Diseño de clases
- Transacciones
- Concurrency
- Error handling
- Testing
- Performance
- Security

Rúbrica Senior

Área	Puntos
C# / .NET	15
ASP.NET Core	15
Angular	15
SQL / EF Core	10
Architecture	15
Coding	15
Testing	5

Área	Puntos
Security	5
Code Review	5
Total	**100**

Interpretación

90–100 — Senior fuerte

Puede diseñar, implementar y defender decisiones técnicas.

80–89 — Senior

Buen candidato; probablemente necesita poca supervisión.

70–79 — Mid/Senior

Tiene experiencia, pero puede necesitar apoyo en arquitectura.

60–69 — Mid

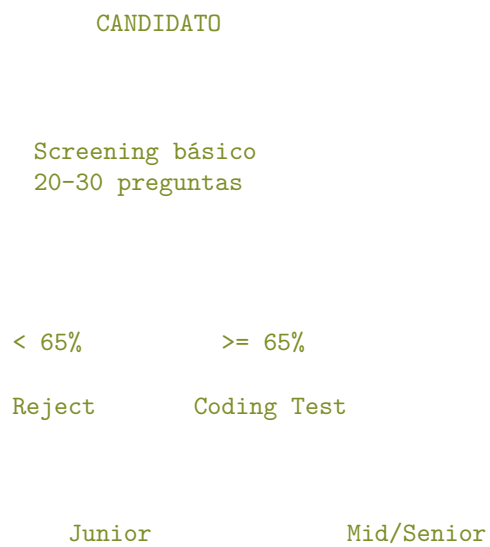
Buen desarrollador, pero no demuestra todavía seniority.

<60 — No Senior

Lo que yo haría para tu proceso

No utilizaría el mismo examen para todos.

Haría este flujo:



`Junior coding` `Advanced coding`

`Architecture interview`

Y **no les daría acceso a las respuestas**. Puedes montar las preguntas teóricas en **Google Forms** y el ejercicio práctico en un repositorio de GitHub.

Si quieres llevar esto a un nivel más profesional, el siguiente paso sería que te prepare **los 3 assessments completos (Junior/Mid/Senior) con unas 30–40 preguntas cada uno, respuestas correctas, explicación de cada respuesta y scoring**, de manera que puedas copiar/pegar directamente a Google Forms.

Definicion de Assessment Técnico

> Si, estoy de acuerdo, esos 3 assesments estarian bien

Perfecto. Te los puedo estructurar como un **paquete de evaluación técnica Full Stack C#/.NET + Angular**, listo para usar.

La propuesta sería:

- **Assessment 1 — Junior:** fundamentos + coding básico.
- **Assessment 2 — Mid-Level:** desarrollo de API, Angular, SQL, EF Core, testing y buenas prácticas.
- **Assessment 3 — Senior:** arquitectura, performance, seguridad, troubleshooting, code review y coding avanzado.
- **30–40 preguntas por nivel.**
- **Respuesta correcta + explicación** para que tú tengas una guía de evaluación.
- **Scoring de 100 puntos** por assessment.
- Preguntas de opción múltiple, verdadero/falso y algunas de respuesta abierta.
- Ejercicios prácticos separados para que puedas evaluar código real.
- Una **rúbrica final** para determinar si el candidato realmente corresponde a Junior, Mid o Senior.

También puedo hacer que las preguntas sean **progresivas y no demasiado obvias**, evitando el típico examen donde alguien puede aprobar simplemente memorizando definiciones.

Te recomiendo además que prepare **dos versiones de cada examen (A y B)**, con preguntas diferentes pero de dificultad equivalente. Así puedes evitar que los candidatos compartan las respuestas entre ellos.

Puedo empezar por el **Assessment Junior completo**, y después hacemos Mid y Senior.

Assessment Técnico — Full Stack Junior

Adelante con el Assessment Junior completo

Perfecto. Aquí tienes el **Assessment Junior completo**, diseñado para evaluar un perfil **Full Stack Junior con C#/.NET + Angular + SQL**. Está pensado para que puedas copiar las preguntas a Google Forms y mantener las respuestas/criterios de evaluación por separado.

Assessment Junior — Full Stack C#/.NET + Angular

Tecnologías: C# / .NET / ASP.NET Core / Entity Framework Core / Angular / TypeScript / SQL Server

Duración recomendada: 75–90 minutos
Puntaje: 100 puntos
Modalidad: Online
Nivel: Junior

Instrucciones para el candidato

Responde todas las preguntas. En las preguntas de selección múltiple, selecciona una sola respuesta salvo que se indique lo contrario.

Para el ejercicio práctico, se evaluará principalmente:

- Correctitud
- Calidad del código
- Organización
- Buenas prácticas
- Capacidad para explicar las decisiones tomadas

No es necesario implementar funcionalidades que no hayan sido solicitadas.

Parte 1 — C# Fundamentals

10 preguntas — 20 puntos

1. ¿Cuál es la diferencia principal entre una `class` y un `struct` en C#?

A. `class` es un value type y `struct` es un reference type.
B. `class` es un reference type y `struct` es un value type.

C. Ambos son reference types.
D. Ambos son value types.

---

2. ¿Qué imprime el siguiente código?

```
int x = 10;
int y = x;
y = 20;

Console.WriteLine(x);
```

A. 10
B. 20
C. 30
D. Error de compilación

---

3. ¿Cuál es el propósito de una interfaz?

A. Crear una instancia automáticamente.
B. Definir un contrato que una clase puede implementar.
C. Reemplazar todas las clases abstractas.
D. Crear una conexión a SQL Server.

---

4. ¿Cuál es una ventaja de utilizar `async/await`?

A. Garantiza que el código utilice múltiples CPUs.
B. Permite trabajar de manera más sencilla con operaciones asíncronas.
C. Convierte automáticamente código síncrono en paralelo.
D. Elimina la necesidad de utilizar `Task`.

---

5. ¿Qué devuelve este método?

```
public async Task<User> GetUserAsync()
{
    ...
}
```

A. `User`
B. `Task`
C. `Task<User>`
D. `void`

---

6. ¿Qué hace LINQ?

A. Permite consultar y transformar colecciones y fuentes de datos.
B. Crea APIs REST automáticamente.
C. Reemplaza C#.
D. Es un ORM.

---

7. ¿Cuál es el resultado?

```
var numbers = new[] { 1, 2, 3, 4, 5 };

var result = numbers
    .Where(x => x > 3)
    .ToList();
```

A. `[1, 2, 3]`
B. `[3, 4, 5]`
C. `[4, 5]`
D. `[1, 2, 3, 4]`

---

8. ¿Para qué se utiliza `try/catch`?

A. Para crear threads.
B. Para manejar excepciones.
C. Para crear interfaces.
D. Para ejecutar consultas SQL.

---

9. ¿Qué problema puede existir en este código?

```
string name = null;

Console.WriteLine(name.Length);
```

A. Stack overflow.
B. Null reference exception.
C. Syntax error.
D. No existe ningún problema.

---

10. ¿Qué principio de programación recomienda que una clase tenga una única responsabilidad?

A. Open/Closed Principle
B. Single Responsibility Principle
C. Dependency Inversion Principle
D. Interface Segregation Principle

Parte 2 — ASP.NET Core / Web API

7 preguntas — 15 puntos

11. ¿Qué es ASP.NET Core?

A. Un lenguaje de programación.
B. Un framework para construir aplicaciones web y APIs.
C. Una base de datos.
D. Un ORM.

---

12. ¿Qué HTTP status code representa normalmente una respuesta exitosa?

A. 200
B. 301
C. 404
D. 500

---

13. ¿Qué status code utilizarías normalmente cuando un recurso no existe?

A. 200
B. 201
C. 404
D. 500

---

14. ¿Qué método HTTP utilizarías normalmente para crear un nuevo recurso?

A. GET
B. POST
C. DELETE
D. OPTIONS

---

15. ¿Qué es Dependency Injection?

A. Un mecanismo para proporcionar las dependencias de una clase desde fuera de la clase.
B. Una herramienta para crear tablas SQL.
C. Una técnica exclusiva de Angular.
D. Un sistema de autenticación.

---

16. ¿Cuál es una responsabilidad típica de un Controller?

A. Renderizar componentes Angular.
B. Recibir requests HTTP y devolver responses.
C. Crear físicamente la base de datos.
D. Compilar TypeScript.

---

17. ¿Qué es middleware en ASP.NET Core?

A. Un componente que participa en el pipeline de procesamiento HTTP.
B. Una tabla de SQL Server.
C. Una clase Angular.
D. Un tipo de DTO.

Parte 3 — Entity Framework Core / SQL

7 preguntas — 15 puntos

18. ¿Qué es Entity Framework Core?

A. Un framework de UI.
B. Un ORM para .NET.
C. Un servidor web.
D. Un lenguaje SQL.

---

19. ¿Qué representa normalmente un `DbContext`?

A. Una conexión/configuración y unidad de trabajo para interactuar con la base de datos.
B. Un Controller.

C. Un componente Angular.
D. Un DTO.

---

20. ¿Qué hace `SaveChangesAsync()`?

A. Ejecuta JavaScript.
B. Persiste cambios pendientes en la base de datos de forma asíncrona.
C. Elimina automáticamente todas las tablas.
D. Ejecuta solamente consultas SELECT.

---

21. ¿Qué hace un `INNER JOIN`?

A. Devuelve registros coincidentes entre las tablas.
B. Devuelve únicamente registros de la primera tabla.
C. Elimina registros duplicados.
D. Combina todas las filas de ambas tablas sin condición.

---

22. ¿Qué hace `ORDER BY`?

A. Agrupa resultados.
B. Ordena resultados.
C. Elimina resultados.
D. Crea índices.

---

23. ¿Qué hace `GROUP BY`?

A. Ordena resultados.
B. Agrupa registros para realizar operaciones agregadas.
C. Elimina tablas.
D. Crea relaciones.

---

24. Considera estas tablas:

```
Customers
---------
Id
Name

Orders
------
```

```
Id
CustomerId
Total
```

¿Cómo obtendrías los nombres de clientes que tienen órdenes?

A.

```
SELECT Name
FROM Customers
INNER JOIN Orders
ON Customers.Id = Orders.CustomerId;
```

B.

```
SELECT Name
FROM Customers
DELETE Orders;
```

C.

```
SELECT Name
FROM Orders;
```

D.

```
SELECT CustomerId
FROM Customers;
```

Parte 4 — Angular / TypeScript

10 preguntas — 20 puntos

25. ¿Qué es Angular?

A. Un lenguaje de programación.
B. Un framework para construir aplicaciones web frontend.
C. Una base de datos.
D. Un ORM.

---

26. ¿Qué lenguaje se utiliza principalmente para desarrollar aplicaciones Angular?

A. C#
B. Java
C. TypeScript
D. SQL

---

27. ¿Qué decorator se utiliza para definir un componente Angular?

A. `@Controller`
B. `@Component`
C. `@Service`
D. `@Angular`

---

28. ¿Para qué se utiliza un Angular Service?

A. Para encapsular lógica reutilizable y/o acceso a datos.
B. Para crear tablas SQL.
C. Para reemplazar todos los componentes.
D. Para ejecutar C#.

---

29. ¿Qué herramienta se utiliza normalmente para realizar HTTP requests en Angular?

A. `HttpClient`
B. `HttpServer`
C. `HttpRequestManager`
D. `AngularHttp`

---

30. ¿Qué es un Observable?

A. Una base de datos.
B. Una representación de una secuencia de valores que pueden emitirse a lo largo del tiempo.
C. Un componente visual.
D. Un Controller.

---

31. ¿Qué hace Angular Routing?

A. Permite manejar la navegación entre diferentes rutas/vistas.
B. Permite conectarse directamente a SQL Server.
C. Ejecuta C# en el navegador.
D. Reemplaza HTTP.

---

32. ¿Para qué se utilizan los Reactive Forms?

A. Para manejar formularios de manera programática y estructurada.
B. Para crear APIs.
C. Para realizar consultas SQL.
D. Para administrar servidores.

---

33. ¿Qué es TypeScript?

A. Un superset de JavaScript que agrega características como tipado estático.
B. Un framework backend.
C. Un ORM.
D. Un lenguaje de consultas.

---

34. ¿Qué hace `ngFor`?

A. Ejecuta código C#.
B. Permite iterar/renderizar elementos de una colección en el template.
C. Hace una petición HTTP automáticamente.
D. Crea un Service.

Parte 5 — Full Stack / Integración

5 preguntas — 10 puntos

35. Tienes:

```
Angular
   ↓
ASP.NET Core API
   ↓
SQL Server
```

¿Dónde debería realizarse normalmente la conexión a SQL Server?

A. Angular
B. ASP.NET Core
C. Browser
D. HTML

---

36. ¿Por qué no deberías conectar Angular directamente a SQL Server?

A. Porque Angular no puede utilizar HTTP.
B. Por seguridad y separación de responsabilidades.
C. Porque SQL Server solamente acepta XML.
D. Porque TypeScript no permite variables.

---

37. ¿Qué formato se utiliza comúnmente para intercambiar datos entre Angular y ASP.NET Core?

A. JSON
B. TXT
C. CSV exclusivamente
D. XML exclusivamente

---

38. Un endpoint devuelve:

`HTTP 400`

¿Qué significa normalmente?

A. Success
B. Bad Request
C. Unauthorized
D. Server Error

---

39. Un endpoint devuelve:

`HTTP 500`

¿Qué indica normalmente?

A. Error del servidor
B. Recurso creado
C. Recurso no encontrado
D. Petición correcta

Parte 6 — Preguntas de razonamiento

5 preguntas — 10 puntos

Estas preguntas pueden utilizarse como respuesta abierta.

40.

Tienes este código:

```
var users = await _context.Users.ToListAsync();

var activeUsers = users
    .Where(x => x.IsActive)
    .ToList();
```

¿Cómo lo mejorarías?

41.

Un usuario presiona el botón “Save” dos veces rápidamente y se crean dos registros.

¿Qué harías para evitar este problema?

42.

Una pantalla Angular tarda 8 segundos en mostrar 10 registros.

¿Qué revisarías primero?

43.

Tu API funciona correctamente en Postman, pero Angular recibe un error relacionado con CORS.

¿Qué investigarías?

44.

Un usuario recibe HTTP 401 al intentar acceder a un endpoint protegido.

¿Qué significa y qué revisarías?

Parte 7 — Coding Challenge

10 puntos

Product API

Construye una API REST utilizando:

```
ASP.NET Core
Entity Framework Core
SQL Server
```

Modelo

```
Product
----------------
Id
Name
Description
Price
Stock
CreatedAt
```

Endpoints requeridos

```
GET    /api/products
GET    /api/products/{id}
POST   /api/products
PUT    /api/products/{id}
DELETE /api/products/{id}
```

Requisitos

La API debe:

- Permitir crear productos.
- Consultar productos.
- Consultar un producto por ID.
- Actualizar productos.
- Eliminar productos.
- Validar que `Name` sea obligatorio.
- Validar que `Price` sea mayor que 0.
- Validar que `Stock` no sea negativo.
- Utilizar Entity Framework Core.
- Utilizar `async/await`.
- Devolver códigos HTTP apropiados.

Bonus

Hasta **5 puntos adicionales**, sin superar 100 puntos:

- DTOs.
- Swagger.

- Unit tests.
- Dependency Injection.
- Manejo global de excepciones.

Clave de respuestas

Esta sección es para el evaluador y **no debe entregarse al candidato**.

#	Respuesta
1	B
2	A
3	B
4	B
5	C
6	A
7	C
8	B
9	B
10	B
11	B
12	A
13	C
14	B
15	A
16	B
17	A
18	B
19	A
20	B
21	A
22	B
23	B
24	A
25	B
26	C
27	B
28	A
29	A
30	B
31	A
32	A
33	A
34	B
35	B

#	Respuesta
36	B
37	A
38	B
39	A

Guía de evaluación — Preguntas abiertas

Pregunta 40

Una buena respuesta sería:

```
var activeUsers = await _context.Users
    .Where(x => x.IsActive)
    .ToListAsync();
```

El candidato debería entender que es mejor realizar el filtro en la base de datos en lugar de traer todos los usuarios a memoria.

Evaluación:

- 0 pts — No identifica el problema.
- 1 pt — Dice que puede optimizarse.
- 2 pts — Propone filtrar antes de `ToListAsync()`.
- 3 pts — Además explica que reduce datos transferidos y uso de memoria.

Pregunta 41

Una respuesta razonable puede incluir:

- Deshabilitar temporalmente el botón.
- Mostrar estado de loading.
- Idempotency.
- Validación backend.
- Constraints en DB cuando corresponda.

Para Junior, basta con que identifique que **la solución no debe depender exclusivamente del frontend**.

Pregunta 42

Esperamos que revise:

- Network requests.

- Backend response time.
- SQL queries.
- Número de requests.
- Payload.
- Rendering.
- Carga innecesaria de datos.

No esperamos necesariamente conocimientos avanzados de profiling.

Pregunta 43

Debe conocer el concepto de **CORS** y entender que es una política de seguridad del navegador relacionada con solicitudes entre diferentes origins.

Debe revisar la configuración de CORS en ASP.NET Core.

Pregunta 44

Debe saber que:

```
401 = Unauthorized
```

Indica normalmente que la autenticación no fue válida o no existe.

Puede revisar:

- JWT.
- Token enviado.
- Expiración.
- Authentication middleware.
- Headers.
- Configuración de autenticación.

Evaluación del Coding Challenge

Criterio	Puntos
API funciona	3
CRUD	2
EF Core	1
Validaciones	1
HTTP status codes	1
async/await	1
Organización/calidad	1
Total	**10**

Señales positivas

- Código limpio.
- Nombres claros.
- Separación razonable de responsabilidades.
- No utiliza lógica de negocio excesiva dentro del Controller.
- Maneja correctamente `404`.
- Utiliza `CreatedAtAction` o equivalente para POST.
- Usa DTOs, aunque no sea obligatorio.
- Utiliza `CancellationToken`, aunque para Junior sería un bonus.

Red flags

- Todo está dentro del Controller.
- SQL hardcodeado sin necesidad.
- No valida inputs.
- Devuelve `200 OK` para cualquier situación.
- Usa `async` pero inmediatamente hace `.Result` o `.Wait()`.
- Expone directamente información sensible.
- No sabe explicar su propio código.

Resultado final

90–100 — Junior fuerte Tiene buenos fundamentos y puede comenzar a trabajar con supervisión normal.

80–89 — Junior Cumple correctamente con las expectativas del nivel.

70–79 — Junior con áreas de mejora Puede ser contratado si las áreas débiles no son críticas.

60–69 — Junior débil Necesitará bastante acompañamiento.

<60 — No recomendado No demuestra todavía los fundamentos necesarios para un puesto Full Stack Junior.

Recomendación de interpretación

No recomiendo utilizar únicamente el porcentaje final.

Por ejemplo:

> 90 puntos pero 20% en Angular

no debería considerarse automáticamente un buen candidato Full Stack.

Yo establecería mínimos:

- **C#/.NET:** 65%
- **Angular:** 60%
- **SQL/EF Core:** 60%
- **Coding:** 60%

Y después utilizaría la entrevista técnica para validar que las respuestas realmente corresponden con el nivel del candidato.

Este sería el **Junior v1**. Para los siguientes assessments mantendría la misma estructura, pero subiría bastante la dificultad: en **Mid** empezaría a evaluar LINQ/EF Core, RxJS, JWT, arquitectura, testing y debugging; en **Senior** pondría escenarios de producción, performance, concurrencia, seguridad, arquitectura y code review.

Assessment Técnico — Full Stack Mid-Level

> Adelante con el Assessment mid completo

Perfecto. Para **Mid-Level** conviene subir la dificultad y, sobre todo, dejar de evaluar únicamente memoria. Este assessment está diseñado para identificar a alguien que pueda **desarrollar features completas con poca supervisión** y detectar problemas comunes de producción.

Assessment Técnico — Full Stack Mid-Level

Tecnologías: C# / .NET / ASP.NET Core / Entity Framework Core / SQL Server / Angular / TypeScript / RxJS

Duración recomendada: 120–150 minutos
Puntaje: 100 puntos
Nivel: Mid-Level

Objetivo

El candidato debe demostrar que puede:

- Desarrollar APIs REST con ASP.NET Core.
- Trabajar correctamente con Entity Framework Core.
- Escribir consultas eficientes.
- Desarrollar aplicaciones Angular.
- Utilizar RxJS.
- Implementar validaciones y manejo de errores.
- Aplicar Dependency Injection.
- Escribir código mantenible.
- Identificar problemas de performance.
- Crear pruebas automatizadas.

- Entender conceptos básicos de seguridad.
- Resolver problemas de integración Full Stack.

Instrucciones para el candidato

Responde todas las preguntas.

En las preguntas de opción múltiple selecciona una sola respuesta, salvo que se indique lo contrario.

En las preguntas abiertas se evaluará principalmente tu razonamiento y no solamente la respuesta final.

Para el ejercicio práctico puedes utilizar documentación oficial de las tecnologías, pero debes poder explicar completamente el código que entregues.

PARTE 1 — C# / .NET

10 preguntas — 15 puntos

1. LINQ y ejecución diferida

Considera:

```
var query = users.Where(x => x.IsActive);

users.Add(new User
{
    Name = "John",
    IsActive = true
});

var result = query.ToList();
```

¿Qué ocurre?

A. `query` contiene solamente los usuarios existentes cuando se creó la variable.
B. `query` se evalúa cuando se ejecuta `ToList()`, por lo que puede incluir el nuevo usuario.
C. El código no compila.
D. `Where()` ejecuta inmediatamente la consulta.

---

2. `IEnumerable` vs `IQueryable`

¿Cuál afirmación es más correcta?

A. `IQueryable` siempre es más rápido que `IEnumerable`.
B. `IEnumerable` solamente funciona con SQL Server.
C. `IQueryable` permite que el proveedor de datos traduzca la expresión, por ejemplo a SQL.
D. No existe diferencia relevante.

---

3. ¿Qué problema existe aquí?

```
var result = users
    .Where(x => x.IsActive)
    .Select(x => GetUserData(x.Id))
    .ToList();
```

Suponiendo que `GetUserData()` realiza una llamada a la base de datos.

A. Ningún problema.
B. Posible N+1 / múltiples llamadas a la base de datos.
C. Error de compilación.
D. Problema exclusivo de Angular.

---

4. Dependency Injection

¿Cuál es la principal ventaja de inyectar:

```
public UserService(IUserRepository repository)
```

en lugar de crear:

```
var repository = new UserRepository();
```

dentro de `UserService`?

A. Hace que el código sea automáticamente más rápido.
B. Permite desacoplamiento y facilita testing/reemplazo de implementaciones.
C. Evita utilizar interfaces.
D. Elimina la necesidad de clases.

---

5. Lifetimes

¿Cuál lifetime utilizarías normalmente para un `DbContext` en una Web API?

A. Singleton
B. Scoped
C. Transient exclusivamente
D. Static

---

6. ¿Qué problema puede producir un `DbContext` Singleton?

A. Ninguno.
B. Problemas de concurrencia, tracking excesivo y ciclo de vida incorrecto.
C. Mejora siempre el rendimiento.
D. Convierte EF Core en NoSQL.

---

7. ¿Cuál es la diferencia principal entre `Task` y `Thread`?

A. Son exactamente lo mismo.
B. `Task` representa una operación de trabajo asíncrona; no necesariamente corresponde a un thread dedicado.
C. `Thread` solamente sirve para SQL.
D. `Task` siempre crea un nuevo thread.

---

8. ¿Qué ventaja tiene `CancellationToken`?

A. Cancela automáticamente todas las requests.
B. Permite propagar una señal para cancelar operaciones en curso cuando ya no son necesarias.
C. Reemplaza `try/catch`.
D. Evita errores HTTP.

---

9. ¿Qué problema tiene este código?

```
var result = GetDataAsync().Result;
```

A. Ninguno.
B. Puede bloquear el thread y contribuir a problemas de deadlock o thread starvation dependiendo del contexto.
C. Siempre es más rápido que `await`.
D. Convierte la operación en paralela.

---

10. ¿Qué principio SOLID representa mejor la idea de depender de abstracciones?

A. SRP
B. OCP
C. DIP
D. ISP

PARTE 2 — ASP.NET Core / Web API

8 preguntas — 15 puntos

11. ¿Qué diferencia existe entre Authentication y Authorization?

A. Son exactamente lo mismo.
B. Authentication determina quién eres; Authorization determina qué puedes hacer.
C. Authorization determina quién eres.
D. Authentication solamente aplica a SQL.

---

12. Un endpoint requiere autenticación y el usuario no proporciona credenciales válidas.

¿Cuál respuesta es normalmente apropiada?

A. 200
B. 201
C. 401
D. 500

---

13. Un usuario está autenticado pero no tiene permisos para ejecutar una operación.

¿Cuál status code corresponde normalmente?

A. 401
B. 403
C. 404
D. 500

---

14. ¿Qué es un DTO?

A. Database Transfer Object
B. Data Transfer Object
C. Dependency Transfer Object
D. Data Table Object

---

15. ¿Por qué evitarías devolver directamente una entidad EF Core desde un Controller?

Selecciona la mejor respuesta.

A. No funciona.
B. Porque puede acoplar el contrato de la API a la persistencia y exponer información que no debería formar parte del contrato.
C. Porque EF Core no permite serializar objetos.
D. Porque solamente funciona con MVC.

---

16. ¿Qué debería hacer una API ante un error inesperado?

A. Devolver siempre el stack trace completo al cliente.
B. Manejarlo de forma centralizada, registrar el error y devolver una respuesta apropiada sin exponer información sensible.
C. Ignorarlo.
D. Devolver siempre HTTP 200.

---

17. ¿Qué ventaja ofrece un Global Exception Handler?

A. Permite centralizar el tratamiento de excepciones.
B. Elimina todas las excepciones.
C. Reemplaza validaciones.
D. Evita utilizar logging.

---

18. ¿Cuál sería una buena práctica para almacenar secretos como passwords de DB o API keys?

A. Hardcodearlos en el código.
B. Guardarlos en Git.
C. Utilizar secret management/configuración segura apropiada para el entorno.
D. Enviarlos desde Angular.

PARTE 3 — Entity Framework Core / SQL

8 preguntas — 15 puntos

19. ¿Qué problema puede existir aquí?

```
var orders = await context.Orders
    .ToListAsync();
```

```
foreach (var order in orders)
{
    order.Customer = await context.Customers
        .FirstAsync(x => x.Id == order.CustomerId);
}
```

A. N+1 queries.
B. SQL injection.
C. Syntax error.
D. Ninguno.

---

20. ¿Cómo mejorarías una consulta de solo lectura?

A. `AsTracking()`
B. `AsNoTracking()`
C. `SaveChanges()`
D. `AttachAll()`

---

21. ¿Qué hace `Include()`?

A. Crea una tabla.
B. Permite cargar relaciones asociadas.
C. Elimina relaciones.
D. Desactiva tracking.

---

22. ¿Cuál consulta es preferible si solamente necesitas Id y Name?

A.

```
var users = await context.Users
    .ToListAsync();
```

B.

```
var users = await context.Users
    .Select(x => new UserDto
    {
        Id = x.Id,
        Name = x.Name
    })
    .ToListAsync();
```

C.

```
var users = context.Users
    .Include(x => x.Orders)
    .Include(x => x.Roles)
    .ToList();
```

D. Todas son equivalentes.

---

23. ¿Qué problema puede provocar traer millones de registros con `ToListAsync()`?

A. Ninguno.
B. Alto consumo de memoria y tiempo de respuesta.
C. El código no compila.
D. SQL Server elimina los datos.

---

24. ¿Qué solución utilizarías para una API que muestra 50 registros por página?

A. Traer todos los registros y hacer `Skip()`/`Take()` después de `ToList()`.
B. Aplicar `Skip()`/`Take()` antes de materializar la consulta.
C. Traer toda la tabla al frontend.
D. Utilizar `Include()`.

---

25. ¿Qué hace un índice en una base de datos?

A. Siempre reduce el tamaño de la tabla.
B. Puede mejorar el rendimiento de determinadas búsquedas, joins y ordenamientos.
C. Sustituye las relaciones.
D. Elimina la necesidad de SQL.

---

26. ¿Qué riesgo existe en construir SQL concatenando directamente input del usuario?

A. N+1
B. SQL Injection
C. CORS
D. Memory leak

PARTE 4 — Angular / TypeScript

10 preguntas — 15 puntos

27. ¿Cuál es una ventaja de utilizar Reactive Forms?

A. Permiten manejar el estado y validación del formulario de forma programática.
B. Solamente funcionan con APIs REST.
C. Reemplazan RxJS.
D. Evitan TypeScript.

---

28. ¿Qué problema existe aquí?

```
ngOnInit() {
  this.userService.getUsers().subscribe(users => {
    this.users = users;
  });
}
```

A. Ninguno necesariamente; depende de cómo esté implementado el Observable y su ciclo de vida.
B. Siempre produce memory leak.
C. Nunca funciona.
D. Angular no permite `subscribe`.

---

29. ¿Qué problema es más evidente aquí?

```
this.http.get('/api/users').subscribe(users => {
  this.http.get('/api/orders').subscribe(orders => {
    this.http.get('/api/products').subscribe(products => {
      ...
    });
  });
});
```

A. SQL Injection
B. Nested subscriptions / código difícil de mantener
C. CORS obligatorio
D. No hay ningún problema

---

30. Si necesitas ejecutar tres requests independientes y esperar a que todas terminen, ¿qué operador podría ser apropiado?

A. `forkJoin`
B. `filter`
C. `map` exclusivamente
D. `tap`

---

31. ¿Qué operador utilizarías frecuentemente para cancelar la request anterior cuando llega un nuevo valor?

A. `switchMap`
B. `reduce`
C. `concat`
D. `scan`

---

32. ¿Qué escenario es apropiado para `debounceTime()`?

A. Búsqueda mientras el usuario escribe.
B. Login solamente.
C. Crear una tabla SQL.
D. Cerrar sesión.

---

33. ¿Qué hace un HTTP interceptor?

A. Intercepta/modifica requests y responses HTTP de forma centralizada.
B. Crea componentes.
C. Modifica SQL Server.
D. Ejecuta C#.

---

34. ¿Para qué podría utilizarse un interceptor?

A. Agregar un JWT a las requests.
B. Manejar determinados errores HTTP.
C. Ambas.
D. Ninguna.

---

35. ¿Qué estrategia puede ayudar a mejorar performance en Angular?

A. Renderizar siempre todos los elementos.
B. Utilizar lazy loading, OnPush, trackBy y/o virtual scrolling según el caso.
C. Hacer todas las requests dentro de `ngOnInit`.
D. Deshabilitar TypeScript.

---

36. ¿Qué problema puede existir al utilizar múltiples subscriptions manuales?

A. Ninguno.
B. Puede complicar el manejo del ciclo de vida y provocar subscriptions activas innecesariamente.
C. Siempre genera SQL Injection.
D. Angular deja de compilar.

PARTE 5 — Full Stack / API Integration

5 preguntas — 10 puntos

37. Un usuario hace login correctamente.

¿Cuál sería una arquitectura razonable?

```
Angular
   ↓
POST /api/auth/login
   ↓
ASP.NET Core
   ↓
Validate credentials
   ↓
  JWT
   ↓
Angular
```

¿Qué debería hacer Angular posteriormente?

A. Guardar y enviar el token de manera apropiada en requests protegidas.
B. Conectarse directamente a SQL Server.
C. Enviar la password en cada request.
D. Crear un usuario directamente en SQL.

---

38. ¿Dónde debería validarse una regla de negocio crítica como:

"No se puede vender más stock del disponible."

A. Solamente Angular.
B. Solamente CSS.
C. Backend y, cuando corresponda, proteger la integridad mediante la base de datos/transacción/concurrencia.
D. Solamente en HTML.

---

39. Angular recibe:

```
HTTP 409 Conflict
```

¿Qué podría representar?

A. Conflicto con el estado actual del recurso.
B. Usuario no autenticado necesariamente.
C. Error de sintaxis TypeScript.
D. Server crash obligatoriamente.

---

40. ¿Por qué una API debería implementar pagination para grandes cantidades de datos?

A. Para que Angular tenga menos TypeScript.
B. Para reducir datos transferidos, memoria y tiempo de respuesta.
C. Para evitar utilizar SQL.
D. Para eliminar HTTP.

---

41. Un endpoint funciona en Postman pero falla desde Angular con CORS.

¿Qué revisarías?

A. Configuración CORS del backend/origin/headers/método.
B. SQL indexes únicamente.
C. Entity Framework únicamente.
D. CSS.

PARTE 6 — Preguntas abiertas / troubleshooting

5 preguntas — 10 puntos

42. Performance de API

Una API tiene este endpoint:

```
[HttpGet]
public async Task<IActionResult> GetProducts()
{
    var products = await _context.Products
        .ToListAsync();

    return Ok(products);
}
```

La tabla contiene 3 millones de productos.

¿Qué problemas identificas y cómo lo mejorarías?

---

43. Angular

Una página realiza la misma request HTTP cinco veces al entrar.

¿Cómo investigarías el problema?

Menciona al menos tres cosas que revisarías.

---

44. Database

Una consulta que anteriormente tardaba 100 ms ahora tarda 8 segundos.

¿Qué revisarías antes de modificar código indiscriminadamente?

---

45. Authentication

Un usuario obtiene `401 Unauthorized` después de que su token funcionaba correctamente durante varias horas.

¿Qué investigarías?

---

46. Production debugging

Un usuario reporta:

> "A veces hago click en Save y aparece dos veces el registro."

¿Cómo investigarías y resolverías el problema?

PARTE 7 — Coding Challenge #1

15 puntos

Product Management API

Crear una API utilizando:

```
ASP.NET Core
Entity Framework Core
SQL Server
```

Entidad

```
Product
--------------------
Id
Name
Description
Price
Stock
CategoryId
CreatedAt
UpdatedAt
```

Requerimientos

Implementar:

```
GET / api / products;
GET / api / products / { id };
POST / api / products;
PUT / api / products / { id };
DELETE / api / products / { id };
```

El GET debe soportar:

```
page;
pageSize;
search;
categoryId;
sortBy;
sortDirection;
```

Ejemplo:

```
GET /api/products?page=1&pageSize=20&search=laptop&sortBy=price&sortDirection=desc
```

Requisitos técnicos

Debe:

- Utilizar async/await.
- Utilizar DTOs.
- Validar input.
- Utilizar Dependency Injection.
- Implementar pagination.
- No devolver entidades directamente.
- Utilizar `AsNoTracking()` en consultas de solo lectura cuando corresponda.
- Manejar `404`.
- Devolver `201 Created` al crear.
- Evitar traer toda la tabla a memoria.
- Manejar errores apropiadamente.

Bonus

- Global exception handling.
- Unit tests.
- Integration tests.
- CancellationToken.
- Filtering dinámico correctamente implementado.
- Swagger/OpenAPI.

PARTE 8 — Coding Challenge #2

15 puntos

Angular Product Screen

Crear una pantalla Angular para consumir:

```
GET / api / products;
```

La pantalla debe mostrar:

```
-----------------------------------------------
Products

Search: [____________]   Category: [ All   ]

-----------------------------------------------
Name          Category       Price       Stock
-----------------------------------------------
Laptop        Electronics    $1200       15
Keyboard      Electronics    $80         32
```

```
Mouse          Electronics    $35         50
-----------------------------------------------

               < 1 2 3 4 5 >
-----------------------------------------------
```

Requisitos

Implementar:

- Angular Component.
- Service.
- HTTP request.
- Reactive Forms para filtros.
- Pagination.
- Loading state.
- Error state.
- Empty state.
- Debounce para búsqueda.
- Manejo apropiado de subscriptions.

Bonus

- `OnPush`.
- `trackBy`.
- Reusable pagination component.
- Tests.
- RxJS bien estructurado.
- URL query parameters sincronizados con los filtros.

CLAVE DE RESPUESTAS

Esta sección es exclusivamente para el evaluador.

#	Respuesta
1	B
2	C
3	B
4	B
5	B
6	B
7	B
8	B
9	B
10	C

#	Respuesta
11	B
12	C
13	B
14	B
15	B
16	B
17	A
18	C
19	A
20	B
21	B
22	B
23	B
24	B
25	B
26	B
27	A
28	A
29	B
30	A
31	A
32	A
33	A
34	C
35	B
36	B
37	A
38	C
39	A
40	B
41	A

GUÍA DE EVALUACIÓN DE PREGUNTAS ABIERTAS

Pregunta 42 — API Performance

Una respuesta Mid-Level debería mencionar:

- Pagination.
- Filtering.
- Sorting.
- `AsNoTracking()`.
- Proyección mediante `Select`.

- Evitar retornar millones de registros.
- Índices adecuados.
- Medir la consulta SQL.

Una respuesta excelente podría mencionar:

- Maximum page size.
- Query plan.
- Caching cuando sea apropiado.
- Cursor/keyset pagination para determinados escenarios.
- Compresión/payload.
- CancellationToken.

Pregunta 43 — Requests duplicadas

Debería revisar:

- Network tab.
- Lifecycle hooks.
- Múltiples componentes solicitando la misma información.
- Duplicación de subscriptions.
- Router navigation.
- Interceptors.
- Services.
- `shareReplay` cuando sea apropiado.
- Estado/cache.

Pregunta 44 — Query lenta

Esperamos:

- Execution plan.
- Índices.
- Cambios en volumen de datos.
- Bloqueos.
- Estadísticas.
- Query generada por EF Core.
- N+1.
- Cambios recientes.
- Carga del servidor.

Un candidato fuerte no debería decir simplemente:

> "Crear un índice."

Primero debería **medir y diagnosticar**.

Pregunta 45 — JWT / 401

Debería revisar:

- Expiración del token.
- Token realmente enviado.
- Authorization header.
- Signature.
- Issuer.
- Audience.
- Clock skew.
- Configuración de authentication.
- Refresh token si existe.

Pregunta 46 — Registro duplicado

Una respuesta sólida debería considerar ambos lados.

Frontend

- Double click.
- Button disabled durante request.
- Estado `isSaving`.
- Requests duplicadas.
- RxJS.

Backend

- Idempotency.
- Validaciones.
- Unique constraints.
- Transactions.
- Concurrency.

Un Mid fuerte debería entender que **deshabilitar el botón en Angular no es suficiente como garantía de integridad**.

RÚBRICA DEL CODING CHALLENGE #1

Criterio	Puntos
CRUD funcionando	3
EF Core correctamente utilizado	2

Criterio	Puntos
Pagination	2
Filtering / Search	1
DTOs	1
Validaciones	1
HTTP status codes	1
Async / Cancellation	1
Error handling	1
Code quality / architecture	2
Total	**15**

Red flags

- `ToList()` antes de `Where()`, `Skip()` o `Take()`.
- Devuelve entidades EF directamente.
- Toda la lógica dentro del Controller.
- No valida inputs.
- No maneja `404`.
- Utiliza `.Result` o `.Wait()` en código async.
- Ignora errores.
- SQL concatenado con input del usuario.
- No puede explicar su implementación.

RÚBRICA DEL CODING CHALLENGE #2

Criterio	Puntos
Component	2
Service / HttpClient	2
Reactive Forms	2
Pagination	2
RxJS	2
Loading/Error/Empty states	2
Code quality	2
Testing / Bonus	1
Total	**15**

SCORING FINAL

Área	Puntos
C# / .NET	15
ASP.NET Core	15
EF Core / SQL	15
Angular / RxJS	15
Full Stack / Integration	10
Troubleshooting	10
Coding API	15
Coding Angular	15
Total bruto	**110**

Para convertirlo a escala de 100:

```
Score = (Puntos obtenidos / 110) × 100
```

Los **bonus** pueden otorgarse dentro de cada sección, pero no deberían permitir que un candidato compense conocimientos fundamentales inexistentes.

INTERPRETACIÓN

90–100 — Mid-Level fuerte Puede trabajar prácticamente de forma independiente en features completas y tiene buen criterio técnico.

80–89 — Mid-Level Cumple las expectativas del nivel.

70–79 — Mid-Level con áreas de mejora Puede funcionar como Mid, pero requiere acompañamiento en algunas áreas.

60–69 — Junior avanzado Tiene conocimientos interesantes, pero todavía no demuestra consistentemente nivel Mid.

<60 — Junior / No recomendado No demuestra los fundamentos necesarios para el puesto.

MÍNIMOS RECOMENDADOS No recomiendo aceptar a alguien solamente por el score global.

Establecería estos mínimos:

Área	Mínimo
C# / .NET	65%

Área	Mínimo
ASP.NET Core	65%
SQL / EF Core	65%
Angular	65%
Coding	60%
Troubleshooting	60%

Un candidato que obtiene 90% global pero solamente 40% en Angular **no debería considerarse automáticamente Full Stack Mid-Level**.

INDICADORES DE UN VERDADERO MID-LEVEL Más allá de las respuestas correctas, durante el assessment busca que el candidato:

- Pregunte antes de implementar cuando existen requisitos ambiguos.
- Piense en errores y casos límite.
- Considere performance.
- Separe responsabilidades.
- No coloque toda la lógica en Controllers.
- Entienda que el frontend no es una barrera de seguridad.
- Sepa utilizar documentación.
- Pueda explicar sus decisiones.
- Sepa identificar cuándo una solución aparentemente correcta puede fallar en producción.

Pregunta final recomendada Después del coding challenge, hazle esta pregunta:

> **"Si tuvieras que poner este código en producción mañana, ¿qué cambiarías antes de hacerlo?"**

Esta pregunta es especialmente útil para un Mid-Level porque permite distinguir entre alguien que **puede hacer que el código funcione** y alguien que empieza a pensar como un desarrollador profesional.

Con esto ya tienes **Junior + Mid-Level** con una estructura comparable. El **Senior** debería ser deliberadamente diferente: menos preguntas de definición y mucho más **system design, arquitectura, concurrencia, seguridad, performance, debugging y code review**.

Assessment Técnico — Full Stack Senior

Adelante con el Assessment Senior completo

Perfecto. Para **Senior** te recomiendo que el assessment no se base principalmente en preguntas de memoria. Un Senior debe demostrar **criterio técnico, diseño, debugging, seguridad, performance y capacidad de tomar decisiones**.

Aquí tienes el assessment completo, manteniendo compatibilidad con el esquema de Junior y Mid.

Assessment Técnico — Full Stack Senior

Tecnologías: C# / .NET / ASP.NET Core / Entity Framework Core / SQL Server / Angular / TypeScript / RxJS

Duración recomendada: 150–180 minutos para el assessment escrito + 60–90 minutos para entrevista técnica/coding review.

Puntaje: 100 puntos

Nivel: Senior

Objetivo

El candidato debe demostrar capacidad para:

- Diseñar y desarrollar aplicaciones Full Stack.
- Tomar decisiones arquitectónicas.
- Identificar problemas de performance.
- Diseñar APIs robustas.
- Manejar concurrencia.
- Diseñar sistemas escalables.
- Aplicar principios SOLID.
- Implementar seguridad.
- Diagnosticar problemas en producción.
- Diseñar soluciones Angular mantenibles.
- Utilizar RxJS correctamente.
- Escribir código testeable.
- Identificar problemas en código existente.
- Explicar trade-offs técnicos.

INSTRUCCIONES PARA EL CANDIDATO

Responde todas las preguntas.

En las preguntas de opción múltiple selecciona una respuesta, salvo cuando se indique lo contrario.

En las preguntas abiertas se evaluará principalmente:

- Razonamiento.

- Trade-offs.
- Capacidad de análisis.
- Experiencia práctica.
- Conocimiento de producción.

Para los ejercicios de código puedes utilizar documentación oficial, pero debes poder explicar completamente las decisiones tomadas.

PARTE 1 — C# / .NET AVANZADO

8 preguntas — 10 puntos

1. Async / Await

Considera:

```
public async Task ProcessAsync()
{
    var users = GetUsersAsync().Result;
    await ProcessUsersAsync(users);
}
```

¿Cuál es el principal problema?

A. `.Result` puede bloquear el thread y provocar problemas de escalabilidad/deadlocks dependiendo del contexto.

B. `await` solamente funciona con `void`.

C. `GetUsersAsync()` no puede devolver una colección.

D. No existe ningún problema.

---

2. Dependency Injection

Tienes:

```
services.AddSingleton<IOrderService, OrderService>();
services.AddScoped<AppDbContext>();
```

`OrderService` depende de `AppDbContext`.

¿Qué problema potencial existe?

A. Ninguno.

B. Un Singleton no debería capturar directamente una dependencia Scoped porque sus lifetimes son incompatibles.

C. Scoped siempre debe ser Singleton.

D. DbContext debe ser Singleton.

---

3. Garbage Collection

¿Cuál afirmación es correcta?

A. GC elimina automáticamente cualquier recurso externo.

B. GC administra memoria administrada, pero recursos como conexiones/streams deben manejarse apropiadamente mediante `using`, `Dispose`, etc.

C. GC elimina inmediatamente cualquier objeto que deja de utilizarse.

D. GC solamente existe en ASP.NET.

---

4. `IDisposable`

¿Por qué implementarías `IDisposable`?

A. Para liberar recursos que requieren disposición determinística.

B. Para mejorar automáticamente el rendimiento de LINQ.

C. Para reemplazar interfaces.

D. Para hacer una clase Singleton.

---

5. Records

¿Cuál es una ventaja de utilizar `record` para determinados modelos?

A. Proporcionan semántica orientada a igualdad por valor y soporte conveniente para objetos inmutables.

B. Siempre tienen mejor rendimiento que una class.

C. Son equivalentes a una tabla SQL.

D. No pueden tener métodos.

---

6. Concurrencia

Dos requests ejecutan simultáneamente:

```
var product = await db.Products
    .FirstAsync(x => x.Id == id);

if (product.Stock > 0)
{
    product.Stock--;

    await db.SaveChangesAsync();
}
```

¿Qué problema potencial existe?

A. Ninguno.

B. Race condition / lost update / overselling bajo concurrencia.

C. SQL Injection.

D. CORS.

---

7. ¿Cuál es una estrategia posible para resolver el problema anterior?

A. Confiar únicamente en Angular.

B. Utilizar mecanismos de concurrencia/transacciones apropiados y/o un update atómico que garantice la condición.

C. Hacer el Controller Singleton.

D. Utilizar `Thread.Sleep()`.

---

8. SOLID

Una clase contiene:

- Validación de órdenes.
- Persistencia.
- Envío de emails.
- Logging.
- Cálculo de descuentos.

¿Qué principio está violando principalmente?

A. SRP

B. OCP

C. LSP

D. ISP

PARTE 2 — ASP.NET CORE / API DESIGN

8 preguntas — 12 puntos

9. Global Exception Handling

¿Por qué es preferible manejar excepciones de forma centralizada?

A. Para evitar todos los errores.

B. Para mantener respuestas consistentes, centralizar logging y evitar duplicación.

C. Para eliminar validaciones.

D. Para ocultar todos los HTTP status codes.

---

10. API Versioning

Una API pública tiene:

`/api/v1/products`

y necesitas cambiar significativamente el contrato.

¿Qué deberías considerar?

A. Romper inmediatamente todos los clientes.

B. Introducir una estrategia de versionado y mantener compatibilidad según los consumidores.

C. Cambiar solamente el nombre de la clase C#.

D. Cambiar SQL Server.

---

11. Idempotency

¿Cuál operación debería ser especialmente considerada para idempotency?

A. Crear una orden/pago donde un retry podría crear dos operaciones.

B. GET de un recurso.

C. Health check.

D. Consulta local.

---

12. Rate Limiting

¿Por qué implementarías rate limiting?

A. Para evitar o limitar abuso, sobrecarga y determinados ataques.

B. Para mejorar automáticamente la calidad del código.

C. Para reemplazar autenticación.

D. Para eliminar SQL indexes.

---

13. Authentication vs Authorization

Un usuario tiene un JWT válido pero intenta acceder a:

`/admin/users`

y no posee el rol requerido.

¿Qué debería ocurrir normalmente?

A. 401

B. 403

C. 404 siempre

D. 500

---

14. ¿Qué problema existe al confiar exclusivamente en autorización del frontend?

A. Ninguno.

B. El frontend puede ser manipulado; las reglas de seguridad deben reforzarse en backend.

C. Angular no permite roles.

D. JWT solamente funciona en frontend.

---

15. ¿Cuál es una buena práctica para API secrets?

A. Guardarlos en código fuente.

B. Guardarlos en GitHub.

C. Utilizar un secret manager/configuración segura.

D. Guardarlos en localStorage.

---

16. ¿Qué información no debería devolverse normalmente en una respuesta de error de producción?

A. Un mensaje genérico.

B. Correlation/request ID.

C. Stack trace interno y secretos/configuración sensible.

D. HTTP status code.

PARTE 3 — EF CORE / SQL / PERFORMANCE

8 preguntas — 14 puntos

17. ¿Qué problema existe?

```
var orders = await db.Orders
    .Include(x => x.Items)
    .Include(x => x.Customer)
    .Include(x => x.Payments)
    .Include(x => x.Shipping)
    .ToListAsync();
```

Si existen cientos de miles de órdenes.

A. `Include` siempre es incorrecto.

B. Puede generar una consulta enorme, grandes resultados/intermedios y problemas de performance.

C. EF Core no soporta Include.

D. SQL Server elimina datos.

---

18. ¿Qué alternativa podría ser apropiada si solo necesitas información específica?

A. Proyección con `Select`.

B. Cargar absolutamente todas las entidades.

C. `ToList()` antes del filtro.

D. `Include` ilimitado.

---

19. ¿Qué problema puede producir `Skip()` con offsets extremadamente grandes?

A. Offset pagination puede volverse costosa porque la DB debe recorrer/saltar muchas filas.

B. Siempre es O(1).

C. Elimina índices.

D. Genera CORS.

---

20. ¿Cuándo considerarías keyset/cursor pagination?

A. Cuando necesitas paginar grandes datasets y quieres evitar offsets muy grandes.

B. Solamente para tablas de dos registros.

C. Nunca.

D. Solamente para Angular.

---

21. ¿Qué herramienta utilizarías para investigar por qué una query SQL está lenta?

A. Execution plan.

B. Cambiar C# aleatoriamente.

C. Reiniciar el frontend.

D. Modificar HTML.

---

22. ¿Qué problema puede producir un índice excesivo?

A. Ninguno.

B. Puede incrementar costos de escritura y almacenamiento.

C. Siempre mejora todas las operaciones.

D. Elimina la necesidad de joins.

---

23. ¿Qué es un deadlock?

A. Una situación donde procesos/transacciones esperan recursos entre sí de forma que ninguno puede continuar.

B. Un error exclusivo de Angular.

C. Una excepción de C# obligatoria.

D. Una forma de cache.

---

24. ¿Qué utilizarías para investigar bloqueos o deadlocks?

A. Herramientas de diagnóstico de SQL Server, logs y execution/deadlock graphs.

B. Cambiar colores del frontend.

C. Reiniciar Angular.

D. Borrar la tabla.

PARTE 4 — ANGULAR / RXJS AVANZADO

8 preguntas — 12 puntos

25. ¿Qué problema existe?

```
searchControl.valueChanges.subscribe(value => {
    this.http.get(`/api/products?search=${value}`)
        .subscribe(result => {
            this.products = result;
        });
});
```

A. Ninguno.

B. Nested/asynchronous request management y ausencia de debounce/cancelación apropiada.

C. SQL Injection obligatoria.

D. Angular no permite valueChanges.

---

26. ¿Qué combinación es apropiada para un search box?

A. `debounceTime + distinctUntilChanged + switchMap`

B. `forkJoin` solamente

C. `reduce`

D. `subscribe` anidados

---

27. ¿Por qué `switchMap` puede ser apropiado para búsquedas?

A. Cancela/descarta la suscripción anterior cuando llega un nuevo valor.

B. Ejecuta siempre todas las requests en paralelo.

C. Convierte Observable en SQL.

D. Crea un componente.

---

28. ¿Cuándo usarías `forkJoin`?

A. Para esperar a que múltiples Observables completen y obtener sus últimos valores.

B. Para cancelar cualquier Observable.

C. Para debounce.

D. Para rendering.

---

29. ¿Qué ventaja puede proporcionar `ChangeDetectionStrategy.OnPush`?

A. Puede reducir trabajo de change detection bajo determinadas arquitecturas.

B. Hace que Angular no detecte ningún cambio.

C. Elimina HTTP.

D. Reemplaza RxJS.

---

30. ¿Cuál es una estrategia adecuada para listas muy grandes?

A. Virtual scrolling / pagination según el caso.

B. Renderizar siempre los 100,000 elementos.

C. Crear 100,000 componentes innecesarios.

D. Desactivar Angular.

---

31. ¿Qué problema puede causar mantener subscriptions largas sin control?

A. Memory leaks o trabajo innecesario.

B. SQL deadlock.

C. C# exception.

D. CORS exclusivamente.

---

32. ¿Qué debería considerar un Senior al diseñar un Angular Service?

A. Responsabilidad, reutilización, estado, caching, lifecycle, error handling y testability.

B. Solamente que tenga muchos métodos.

C. Que toda la aplicación utilice un único Service gigante.

D. Que acceda directamente al DOM siempre.

PARTE 5 — SECURITY

6 preguntas — 10 puntos

33. ¿Dónde debería validarse autorización para una operación crítica?

A. Solamente en Angular.

B. En backend.

C. Solamente en HTML.

D. En CSS.

---

34. ¿Qué es SQL Injection?

A. Una técnica donde input malicioso altera una consulta SQL.

B. Un tipo de CORS.

C. Un error de Angular.

D. Un mecanismo de autenticación.

---

35. ¿Cómo reducirías riesgo de SQL Injection?

A. Concatenar strings cuidadosamente.

B. Utilizar queries parametrizadas/ORM correctamente.

C. Desactivar SQL Server.

D. Validar solamente en Angular.

---

36. ¿Qué problema tiene guardar información sensible en `localStorage`?

A. Puede estar expuesta a JavaScript ejecutado en el contexto de la aplicación, por ejemplo ante XSS.

B. Es imposible acceder.

C. Es más seguro que cualquier alternativa automáticamente.

D. SQL Server la protege.

---

37. ¿Qué es XSS?

A. Cross-Site Scripting.

B. Cross SQL Server.

C. XML Security System.

D. External Service Session.

---

38. ¿Qué estrategia ayuda contra XSS?

A. Output encoding/sanitization apropiados y evitar insertar contenido no confiable como HTML.

B. Confiar únicamente en TypeScript.

C. Guardar todo en localStorage.

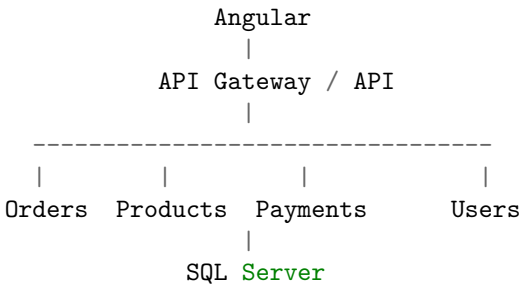
D. Deshabilitar HTTPS.

PARTE 6 — SYSTEM DESIGN

10 puntos

39. Diseño de sistema

Diseña una plataforma de e-commerce:



Requisitos:

- 1 millón de usuarios.
- 100,000 órdenes diarias.
- Picos de tráfico 10x.
- Pagos externos.
- Inventario.
- Notificaciones.
- Auditoría.
- Alta disponibilidad.

Explica:

A. Arquitectura

¿Qué componentes utilizarías?

B. Database

¿Cómo organizarías las principales entidades?

C. Caching

¿Qué información cachearías?

D. Messaging

¿Dónde utilizarías eventos/mensajería?

E. Reliability

¿Cómo manejarías fallos de servicios externos?

F. Observability

¿Cómo detectarías problemas?

G. Security

¿Cómo protegerías APIs y datos?

H. Scalability

¿Qué componentes escalarías horizontalmente?

PARTE 7 — PRODUCTION TROUBLESHOOTING

10 puntos

40. API lenta

Producción reporta:

> "La API de órdenes pasó de 200 ms a 8 segundos."

¿Qué investigarías?

El candidato debería estructurar una estrategia de diagnóstico.

Esperamos considerar:

- Logs.
- Métricas.
- Traces.
- Database performance.
- Query plans.
- Blocking/deadlocks.
- External APIs.
- CPU/memory.
- Connection pool.
- Network.
- Cambios recientes.
- Cache.
- Deployment reciente.

---

41. Memory leak

Después de varias horas el proceso ASP.NET Core consume cada vez más memoria.

¿Qué investigarías?

---

42. Angular lento

Una pantalla tiene 5,000 registros y se vuelve lenta después de agregar filtros.

¿Cómo investigarías?

---

43. Requests duplicadas

Una aplicación Angular está ejecutando la misma API 10 veces.

¿Cómo encontrarías el origen?

---

44. Error intermitente

Una API devuelve HTTP 500 aproximadamente 1 de cada 100 requests.

¿Cómo diagnosticarías el problema?

PARTE 8 — CODE REVIEW BACKEND

8 puntos

Considera:

```
[HttpPost]
public async Task<IActionResult> CreateOrder(Order order)
{
    var product = await _context.Products
        .FirstAsync(x => x.Id == order.ProductId);

    if (product.Stock >= order.Quantity)
    {
        product.Stock -= order.Quantity;

        await _context.SaveChangesAsync();

        await _emailService.SendAsync(order.CustomerEmail);

        return Ok(order);
    }

    return Ok("No stock");
}
```

45. Identifica los problemas

El candidato debe identificar tantos como sea posible.

Problemas esperados:

- Entity binding directamente desde HTTP.
- Ausencia de DTO.
- Validación insuficiente.
- `FirstAsync()` puede producir excepción.
- `200 OK` para "No stock".
- No existe status semántico apropiado.
- Race condition sobre stock.
- No hay transacción explícita cuando el proceso lo requiere.
- Email enviado dentro del flujo crítico.
- Si email falla después de DB commit, el comportamiento puede quedar inconsistente.
- No hay idempotency.
- Falta CancellationToken.
- No hay logging.
- No existe autorización visible.
- No hay manejo centralizado de excepciones.
- Falta respuesta `Created`.
- Falta modelado adecuado de Order/OrderItems.
- Falta validación de Customer.
- Posible problema de concurrencia.
- Posibles problemas de performance.

PARTE 9 — CODE REVIEW ANGULAR

8 puntos

Considera:

```
export class OrderComponent {

  orders: any[] = [];

  ngOnInit() {
    this.loadOrders();
  }

  loadOrders() {
    this.http.get<any[]>('/api/orders').subscribe(data => {
      this.orders = data;
    });
  }
```

```
  delete(id: number) {
    if (confirm('Delete?')) {
      this.http.delete('/api/orders/' + id).subscribe(() => {
        this.loadOrders();
      });
    }
  }
}
```

46. Identifica problemas y propone mejoras.

Esperamos considerar:

- `any`.
- HTTP directamente en Component.
- Service debería encapsular acceso a API.
- Manejo de errores.
- Loading state.
- Confirmation UI.
- Refresh completo después de delete.
- Lifecycle/subscription management.
- Tipado.
- Optimistic/pessimistic update según el caso.
- RxJS.
- Testability.
- UI state.
- Error feedback.
- Potential duplicate requests.
- Reusabilidad.

PARTE 10 — CODING CHALLENGE

15 puntos

Order Processing API

Construye una API para procesar órdenes.

Modelo

```
Order
-------------------------------
Id
CustomerId
```

```
CreatedAt
Status
Total

OrderItem
---------------------------------
Id
OrderId
ProductId
Quantity
UnitPrice
```

Requisitos

Endpoint:

`POST /api/orders`

Request:

```
{
  "customerId": 100,
  "items":
    [{ "productId": 10, "quantity": 2 }, { "productId": 20, "quantity": 1 }],
}
```

El sistema debe:

1. Validar customer.
2. Validar productos.
3. Validar stock.
4. Obtener precios actuales.
5. Calcular total.
6. Crear la orden.
7. Reducir inventario.
8. Mantener consistencia transaccional.
9. Devolver el resultado.

Consideraciones

El candidato debe explicar cómo manejaría:

- Dos requests comprando simultáneamente el mismo producto.
- Retry del mismo request.
- Error durante persistencia.
- Error de un servicio externo.
- Timeout.
- Cancellation.
- Logging.

- Idempotency.

PARTE 11 — CODING CHALLENGE ANGULAR

8 puntos

Construye una pantalla:

```
Orders
-------------------------------------------------

Search: [_________________]

Status: [All  ]

-------------------------------------------------
Order     Customer      Date        Status   Total
-------------------------------------------------
10001     John          09/01       Paid     $120
10002     Mary          09/02       Pending  $80
10003     Bob           09/02       Shipped  $250
-------------------------------------------------

                < 1 2 3 4 5 >
```

Requisitos

Implementar:

- Angular Component.
- Service.
- Typed models.
- Reactive Forms.
- Search.
- Status filter.
- Pagination.
- Debounce.
- Loading.
- Error handling.
- Empty state.

La búsqueda debe utilizar una estrategia RxJS apropiada.

Ejemplo conceptual:

```
valueChanges
    ↓
```

```
    debounceTime
        ↓
distinctUntilChanged
        ↓
    switchMap
        ↓
      HTTP
        ↓
       UI
```

CLAVE DE RESPUESTAS

Exclusivamente para el evaluador.

#	Respuesta
1	A
2	B
3	B
4	A
5	A
6	B
7	B
8	A
9	B
10	B
11	A
12	A
13	B
14	B
15	C
16	C
17	B
18	A
19	A
20	A
21	A
22	B
23	A
24	A
25	B
26	A
27	A
28	A
29	A

#	Respuesta
30	A
31	A
32	A
33	B
34	A
35	B
36	A
37	A
38	A

RÚBRICA SYSTEM DESIGN

Excelente — 9–10 puntos

El candidato:

- Define claramente componentes.
- Explica trade-offs.
- Considera escalabilidad.
- Considera fallos.
- Utiliza caching de manera razonada.
- Considera messaging/event-driven donde corresponde.
- Habla de observabilidad.
- Considera seguridad.
- Entiende consistencia vs disponibilidad.
- Considera external services y retries.
- No intenta solucionar todo con microservices.

Bueno — 7–8 puntos

Tiene arquitectura razonable pero puede omitir algunos aspectos de producción.

Regular — 5–6 puntos

Conoce componentes pero responde de forma superficial.

Débil — <5 puntos

Piensa principalmente en implementar endpoints sin considerar operación, escalabilidad o fallos.

RÚBRICA CODE REVIEW BACKEND

Criterio	Puntos
Identificación de problemas	3
Concurrencia	1
Error handling	1
Security / validation	1
Architecture	1
Reliability	1
Total	**8**

Un Senior fuerte debería identificar **al menos 8–10 problemas diferentes** en el código.

RÚBRICA CODE REVIEW ANGULAR

Criterio	Puntos
Architecture	2
RxJS / subscriptions	1
Error handling	1
Typing	1
Performance	1
Maintainability	1
Testing	1
Total	**8**

RÚBRICA CODING BACKEND

Criterio	Puntos
Correctitud funcional	3
Architecture / separation	2
EF Core / DB	2
Concurrency	2
Validation / security	1
Error handling	1
Testing	1
Idempotency / reliability	1
Code quality	2
Total	**15**

RÚBRICA CODING ANGULAR

Criterio	Puntos
Funcionalidad	2
Component / Service separation	1
RxJS	1
Forms	1
Pagination/filter	1
Error/loading states	1
Code quality / typing	1
Total	**8**

SCORING FINAL

El assessment suma:

Área	Puntos
C# / .NET	10
ASP.NET Core / API	12
EF Core / SQL / Performance	14
Angular / RxJS	12
Security	10
System Design	10
Production Troubleshooting	10
Backend Code Review	8
Angular Code Review	8
Backend Coding	15
Angular Coding	8
Total bruto	**117**

Convertir a porcentaje:

```
Score = (Puntos obtenidos / 117) × 100
```

INTERPRETACIÓN DEL RESULTADO

90–100 — Senior fuerte Tiene conocimientos técnicos sólidos y demuestra criterio para tomar decisiones de arquitectura y producción.

80–89 — Senior Cumple claramente las expectativas de Senior.

70–79 — Mid/Senior Tiene experiencia, pero presenta áreas importantes que deberían validarse durante la entrevista.

60–69 — Mid-Level fuerte Puede ser un excelente Mid, pero todavía no demuestra consistentemente seniority.

<60 — No Senior No demuestra el nivel técnico esperado.

MÍNIMOS RECOMENDADOS

No recomiendo utilizar solamente el score global.

Establecería mínimos:

Área	Mínimo
C# / .NET	70%
ASP.NET Core	70%
SQL / EF Core	70%
Angular	65%
Security	60%
Architecture	65%
Coding	65%
Troubleshooting	65%

Un candidato con excelente C# pero muy bajo Angular **no debería considerarse automáticamente Senior Full Stack**.

RED FLAGS DE SENIOR

Estas señales deberían pesar mucho durante la evaluación:

1. "Lo arreglo en Angular" Si una regla de seguridad o negocio crítica se implementa solamente en frontend.

2. "Más microservices" Si la respuesta para cualquier problema arquitectónico es crear microservicios sin explicar por qué.

3. "Le pongo un índice" Ante cualquier problema de SQL, sin analizar execution plan ni medir.

4. "Uso Singleton para mejorar performance" Especialmente si lo aplica indiscriminadamente a componentes stateful como `DbContext`.

5. "Hago retry" Sin hablar de idempotency, especialmente para operaciones financieras o de creación.

6. "Deshabilito el botón" Como única solución a duplicación de órdenes.

7. "Manejo todo con try/catch" Sin estrategia de error handling, logging y propagación.

8. No puede explicar su propio código Esta debería ser una señal importante independientemente del score.

PREGUNTAS FINALES PARA LA ENTREVISTA

Después del assessment, recomiendo hacer estas preguntas verbalmente.

Pregunta 1

> "Cuéntame sobre una decisión técnica que hayas tomado y que posteriormente hayas tenido que cambiar."

Busca:

- humildad técnica
- experiencia real
- aprendizaje
- capacidad de reconocer errores

Pregunta 2

> "¿Cuál ha sido el problema de producción más difícil que hayas investigado?"

Busca:

- debugging sistemático
- observabilidad
- logs
- métricas
- capacidad de aislar problemas

Pregunta 3

> "Si heredas una aplicación Angular + .NET que funciona pero tiene mucho código legacy, ¿qué harías durante los primeros 30 días?"

Busca:

- no reescribir todo inmediatamente
- identificar riesgos
- medir
- priorizar
- tests
- deuda técnica
- quick wins

Pregunta 4

> "¿Cuándo NO utilizarías microservices?"

Esta es especialmente útil para identificar madurez arquitectónica.

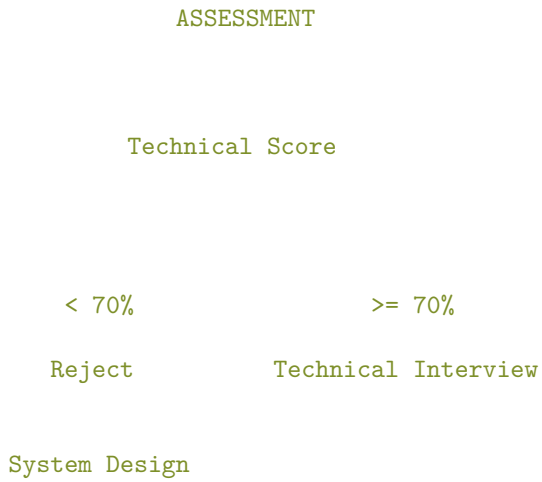
Pregunta 5

> "¿Qué parte de tu código del assessment cambiarías si tuvieras que ponerlo mañana en producción?"

Un candidato Senior debería poder criticar su propia solución.

DECISIÓN FINAL

Yo combinaría el resultado de esta manera:



```
Code Review
```

```
Final Score
```

Y para un candidato Senior, **no contrataría basándome solamente en el examen**. El assessment debe ser el filtro técnico; la entrevista debe validar que el conocimiento es realmente suyo y que puede defender sus decisiones.

Con esto quedan definidos los **tres niveles — Junior, Mid y Senior — bajo la misma filosofía de evaluación**, pero con una diferencia clara de seniority: **Junior = fundamentos, Mid = implementación y troubleshooting, Senior = diseño, trade-offs y producción**.

Prueba de Código Full Stack — Mid-Level

> Ahora bien puedes generar una prueba de codigo fullstack c# angular para un nivel Medio?

Sí. Para un **Mid-Level Full Stack C#/.NET + Angular**, te recomiendo una prueba de código de **2–3 horas**, basada en un escenario que se parezca a trabajo real. La clave es que no sea simplemente "haz un CRUD", sino que permita evaluar **arquitectura, calidad de código, EF Core, SQL, Angular, RxJS, validaciones y manejo de errores**.

Te dejo una prueba lista para entregar al candidato, **sin las respuestas**. Después, si quieres, puedo prepararte la **versión del evaluador con solución esperada y rúbrica de 100 puntos**.

Prueba de Código Full Stack Mid-Level — C#/.NET + Angular

Prueba de Código Full Stack — Mid-Level

C#/.NET + Angular + SQL Server

Nivel: Mid-Level
Tiempo recomendado: 2–3 horas
Modalidad: Take-home / Coding Assessment
Puntaje: 100 puntos

---

1. Objetivo

El objetivo de esta prueba es evaluar tu capacidad para desarrollar una funcionalidad Full Stack utilizando:

- C#
- ASP.NET Core Web API
- Entity Framework Core
- SQL Server
- Angular
- TypeScript
- RxJS

Se evaluará no solamente que la solución funcione, sino también:

- Calidad del código.
- Arquitectura.
- Separación de responsabilidades.
- Validaciones.
- Manejo de errores.
- Performance.
- Diseño de API.
- Experiencia con Angular y RxJS.
- Capacidad de tomar decisiones técnicas.

---

2. Escenario

Una empresa necesita una aplicación interna para administrar productos y órdenes.

Los usuarios podrán:

- Consultar productos.
- Buscar productos.
- Crear productos.
- Modificar productos.
- Crear órdenes.
- Consultar órdenes.
- Consultar el detalle de una orden.

La aplicación tendrá la siguiente arquitectura:

```
Angular

Product Management
Order Management

HTTP/JSON

ASP.NET Core API
```

Controllers
Services
EF Core

SQL Server

---

3. Modelo de datos

Product

```
Product
-------------------------
Id
Name
Description
Price
Stock
CategoryId
IsActive
CreatedAt
UpdatedAt
```

Category

```
Category
-------------------------
Id
Name
IsActive
```

Order

```
Order
-------------------------
Id
CustomerName
CreatedAt
Status
Total
```

OrderItem

```
OrderItem
-------------------------
Id
OrderId
ProductId
Quantity
UnitPrice
Subtotal
```

Relaciones:

```
       Category

           1

           *
       Product

           1

           *
     OrderItem

           *

           1
       Order
```

---

4. Backend

Crear una API utilizando:

```
ASP.NET Core Web API
Entity Framework Core
SQL Server
```

Puedes utilizar Code First o Database First.

---

5. Product API

Implementar los siguientes endpoints.

Obtener productos

`GET /api/products`

Debe soportar:

```
page
pageSize
search
categoryId
sortBy
sortDirection
```

Ejemplo:

```
GET /api/products?page=1&pageSize=10&search=laptop&categoryId=2&sortBy=price&sortDirection=a
```

La respuesta debe incluir información de paginación.

Ejemplo:

```
  {
    "items":
      [
        {
          "id": 1,
          "name": "Laptop",
          "description": "Business laptop",
          "price": 1200,
          "stock": 15,
          "categoryId": 2,
        },
      ],
    "page": 1,
    "pageSize": 10,
    "totalItems": 25,
    "totalPages": 3,
  }
```

Obtener producto

```
GET / api / products / { id };
```

Debe devolver:

`200 OK`

si existe.

Si no existe:

404 Not Found

Crear producto

POST / api / products;

Request:

```
{
  "name": "Mechanical Keyboard",
  "description": "RGB keyboard",
  "price": 120,
  "stock": 50,
  "categoryId": 2,
}
```

Validaciones:

- name requerido.
- name no debe superar 200 caracteres.
- price debe ser mayor que 0.
- stock no puede ser negativo.
- categoryId debe existir.
- La categoría debe estar activa.

Si la operación es exitosa:

201 Created

Actualizar producto

PUT /api/products/{id}

Debe permitir modificar:

- Name
- Description
- Price
- Stock
- CategoryId
- IsActive

Debe validar los mismos criterios del POST.

Eliminar producto

`DELETE /api/products/{id}`

No es obligatorio eliminar físicamente el registro.

Puedes implementar:

`IsActive = false`

como soft delete.

Explica en la documentación qué decisión tomaste.

---

6. Category API

Implementar:

`GET /api/categories`

Debe devolver únicamente categorías activas.

Ejemplo:

```
[{ "id": 1, "name": "Electronics" }, { "id": 2, "name": "Office" }]
```

---

7. Order API

Implementar:

`POST /api/orders`

Request:

```
{
  "customerName": "John Smith",
  "items":
    [{ "productId": 1, "quantity": 2 }, { "productId": 5, "quantity": 1 }],
}
```

El backend debe:

1. Validar que exista el producto.
2. Validar que el producto esté activo.
3. Validar que exista stock suficiente.
4. Obtener el precio actual del producto.
5. Calcular el subtotal.
6. Calcular el total.
7. Crear la orden.
8. Crear los OrderItems.
9. Actualizar el stock.

El precio enviado por el cliente **no debe utilizarse como precio final**.

El backend debe utilizar el precio almacenado en la base de datos.

---

8. Consistencia

Considera este escenario:

```
Product Stock = 5

Request A → compra 4
Request B → compra 3
```

Ambas requests llegan prácticamente al mismo tiempo.

La aplicación no debe permitir que el stock termine siendo inconsistente.

Explica brevemente en el README cómo manejaste este escenario.

No es necesario implementar una solución extremadamente compleja, pero sí debes demostrar que entendiste el problema de concurrencia.

---

9. Consultar órdenes

Implementar:

```
GET /api/orders
```

Debe soportar:

```
page
pageSize
status
search
```

Ejemplo:

```
GET /api/orders?page=1&pageSize=20&status=Completed&search=John
```

La respuesta debe incluir paginación.

---

10. Obtener detalle de una orden

Implementar:

```
GET /api/orders/{id}
```

Ejemplo:

```
{
  "id": 1001,
  "customerName": "John Smith",
  "createdAt": "2026-09-04T10:30:00",
  "status": "Completed",
  "total": 1450,
  "items":
    [
      {
        "productId": 1,
        "productName": "Laptop",
        "quantity": 1,
        "unitPrice": 1200,
        "subtotal": 1200,
      },
      {
        "productId": 5,
        "productName": "Keyboard",
        "quantity": 1,
        "unitPrice": 250,
        "subtotal": 250,
      },
    ],
}
```

---

11. Requisitos de Backend

La solución debe:

- Utilizar Dependency Injection.
- Utilizar async/await.
- Utilizar DTOs.
- Separar responsabilidades razonablemente.
- Utilizar Entity Framework Core.
- Utilizar LINQ.
- Evitar traer datos innecesarios.
- Implementar pagination en database level.
- Implementar validaciones.
- Utilizar HTTP status codes apropiados.
- Manejar errores.
- Evitar SQL injection.
- Evitar devolver directamente entidades EF Core desde los Controllers.

---

12. Angular

Crear una aplicación Angular que consuma la API.

Debe contener al menos:

```
Products
Orders
```

---

13. Product List

Crear una pantalla:

```
--------------------------------------------------
Products
--------------------------------------------------

Search: [____________________]

Category: [ All Categories  ]

Sort:
[ Name  ] [ Ascending  ]

                        [Add Product]

--------------------------------------------------
Name              Category       Price       Stock
--------------------------------------------------
Laptop            Electronics    $1200       15
Keyboard          Electronics    $120         50
Mouse             Electronics    $35         100
--------------------------------------------------

                 < 1 2 3 4 5 >

--------------------------------------------------
```

Debe implementar:

- Search.
- Category filter.
- Sorting.
- Pagination.
- Loading indicator.
- Error state.
- Empty state.

---

14. Product Form

Crear un formulario para:

```
Create Product
```

Campos:

```
Name
Description
Price
Stock
Category
```

Debe utilizar **Reactive Forms**.

Validaciones:

```
Name        required
Price       required + > 0
Stock       required + >= 0
Category    required
```

Mostrar mensajes de validación apropiados.

---

15. Edit Product

El mismo formulario debe permitir editar un producto.

Al seleccionar:

```
Edit
```

debe cargar la información existente.

El formulario debe distinguir entre:

```
Create
```

y:

```
Edit
```

---

16. Orders

Crear una pantalla:

```
--------------------------------------------------
Orders
--------------------------------------------------

Search: [_________________]

Status: [ All   ]

--------------------------------------------------
Order     Customer       Date        Status   Total
--------------------------------------------------
1001      John Smith     09/01       Completed $1200
1002      Mary Smith     09/02       Pending   $250
1003      Bob Johnson    09/03       Completed $800
--------------------------------------------------

                  < 1 2 3 4 5 >

--------------------------------------------------
```

Debe permitir:

- Buscar.
- Filtrar por status.
- Paginar.
- Ver detalle.

---

17. Create Order

Crear una pantalla/formulario que permita:

```
Customer Name
```

y agregar productos:

```
Product         Quantity        Price       Subtotal
--------------------------------------------------
Laptop          [- 1 +]         $1200       $1200
Keyboard        [- 2 +]         $120        $240
--------------------------------------------------

                                Total: $1440

                            [Create Order]
```

El frontend debe:

- Permitir agregar productos.

- Modificar cantidades.
- Mostrar total.
- Validar cantidades.
- Evitar cantidades <= 0.
- Mostrar errores del backend.
- Mostrar loading durante el proceso.

El backend debe ser quien determine el precio final.

---

18. RxJS

La búsqueda de productos y órdenes debe implementar un comportamiento similar a:

```
User types
    ↓
debounceTime
    ↓
distinctUntilChanged
    ↓
switchMap
    ↓
HTTP request
    ↓
Update UI
```

No se recomienda realizar requests HTTP innecesarias en cada tecla presionada.

---

19. Angular Architecture

La aplicación debe separar razonablemente:

```
Components
Services
Models
Forms
```

El acceso HTTP debe realizarse desde Services y no directamente desde los componentes.

Se espera utilizar TypeScript con tipos adecuados.

Evita utilizar:

```
any
```

sin una razón justificada.

---

20. Manejo de errores

La aplicación debe manejar al menos:

```
400 Bad Request
404 Not Found
409 Conflict
500 Internal Server Error
```

Cuando sea apropiado.

Debe mostrar al usuario mensajes comprensibles.

No es necesario mostrar información técnica interna del backend.

---

21. Requisitos adicionales

Implementar al menos:

Backend

- Swagger/OpenAPI.
- Global exception handling.
- DTOs.
- Dependency Injection.
- Entity Framework Core migrations.

Angular

- Reactive Forms.
- RxJS.
- Loading state.
- Error state.
- Empty state.

---

22. Testing

No es necesario tener cobertura completa.

Sin embargo, se espera al menos:

Backend 3–5 unit tests para lógica importante.

Por ejemplo:

`CreateOrder_WithValidProducts_ShouldCreateOrder`

`CreateOrder_WithInsufficientStock_ShouldFail`

`CreateOrder_WithInactiveProduct_ShouldFail`

Angular Al menos 2 tests.

Por ejemplo:

`ProductForm_ShouldRequireName`

`ProductForm_ShouldRejectNegativePrice`

---

23. Bonus

Hasta 10 puntos adicionales.

Se pueden obtener por:

- Docker / Docker Compose.
- Integration tests.
- Authentication/JWT.
- Refresh tokens.
- Optimistic concurrency.
- Centralized logging.
- CI/CD pipeline.
- Reusable Angular components.
- OnPush.
- Strong typing.
- CancellationToken.
- Good README/documentation.

---

24. README

Incluye un README explicando:

Requirements

Qué necesita instalar el evaluador.

Installation

Pasos para ejecutar:

```
Backend
Frontend
Database
```

Database

Cómo crear/migrar la base de datos.

Configuration

Variables/configuración necesarias.

Architecture

Explica brevemente la estructura de tu solución.

Decisions

Explica al menos:

1. ¿Por qué elegiste tu estructura de backend?
2. ¿Cómo manejas errores?
3. ¿Cómo manejas pagination?
4. ¿Cómo manejas concurrencia de stock?
5. ¿Cómo manejas el estado de Angular?
6. ¿Qué mejorarías si tuvieras más tiempo?

---

25. Entregables

El candidato debe entregar:

```
/project
   /backend
   /frontend
   README.md
```

Opcional:

```
docker-compose.yml
```

No es necesario entregar la carpeta:

```
node_modules
```

ni archivos generados innecesarios.

---

26. Criterios de evaluación

La solución será evaluada de acuerdo con:

Área	Puntos
Backend / ASP.NET Core	15
C# / Code Quality	10
EF Core / SQL	10
API Design	10
Angular	15
RxJS	10
Full Stack Integration	10
Validation / Error Handling	5
Testing	5
Documentation	5
Performance / Security	5
Total	**100**

Los bonus pueden utilizarse para distinguir candidatos con soluciones particularmente buenas, pero no deben compensar deficiencias importantes en funcionalidades obligatorias.

---

27. Evaluación del nivel

90–100 Strong Mid-Level

La solución demuestra independencia, buen criterio técnico y conocimiento sólido de Full Stack.

80–89 Mid-Level

Cumple adecuadamente con las expectativas.

70–79 Mid-Level con áreas de mejora

Tiene capacidad, pero presenta algunas debilidades técnicas.

60–69 Junior avanzado

Puede desarrollar features, pero todavía requiere supervisión considerable.

<60 No alcanza Mid-Level

Presenta deficiencias importantes en fundamentos o implementación.

---

28. Restricciones

No es necesario implementar:

- Microservices.
- Kubernetes.
- Cloud infrastructure.
- Event sourcing.
- CQRS complejo.
- Redis.
- Kafka/RabbitMQ.

La evaluación busca medir **calidad de ingeniería**, no cantidad de tecnologías utilizadas.

Una solución sencilla, bien diseñada y correctamente implementada será mejor evaluada que una arquitectura excesivamente compleja sin justificación.

---

29. Pregunta final

En el README responde:

> **Si esta aplicación fuera a utilizarse por 100,000 usuarios y tuvieras una semana adicional para mejorarla, ¿qué tres cosas cambiarías primero y por qué?**

Explica tus prioridades y el impacto esperado de cada cambio.

Mi recomendación

Esta prueba está bastante bien para un **Mid real**, porque obliga al candidato a demostrar varias cosas que un simple CRUD no detecta: **paginación en DB, DTOs, RxJS, concurrencia de inventario, validación, separación de responsabilidades y manejo de errores**.

Para usarla en un proceso de contratación, el siguiente paso que haría es preparar una **"versión del entrevistador"** con:

- solución de referencia completa en **ASP.NET Core + EF Core + SQL Server + Angular**;
- qué debería hacer un candidato Mid;
- errores típicos y *red flags*;
- rúbrica detallada de los **100 puntos**;
- ejemplos de solución **Junior / Mid / Strong Mid**;
- preguntas para revisar el código durante una entrevista de 30 minutos.

Eso te permitiría evaluar el repositorio de cada candidato de forma bastante objetiva.

> Ahora bien requiero una pruab de codigo de fullstack c# y angular a nivel senior.

Sí. Para **Senior Full Stack C#/.NET + Angular** cambiaría bastante el enfoque respecto al Mid: menos CRUD y más **diseño, concurrencia, resiliencia, seguridad, performance, observabilidad y decisiones técnicas**.

Te propongo una prueba de **4–5 horas**, donde el candidato no necesariamente tiene que implementar absolutamente todo. Lo importante es que las partes críticas estén bien resueltas y pueda **defender sus decisiones**.

Prueba de Código Full Stack — Senior

C#/.NET + Angular + SQL Server

Nivel: Senior
Tiempo recomendado: 4–5 horas
Modalidad: Take-home
Puntaje: 100 puntos + 10 puntos bonus

---

1. Objetivo

El objetivo de esta prueba es evaluar la capacidad del candidato para diseñar e implementar una solución Full Stack que pueda evolucionar hacia un sistema real de producción.

Se evaluarán especialmente:

- C# avanzado.
- ASP.NET Core.
- Entity Framework Core.
- SQL Server.
- Diseño de APIs.
- Arquitectura.
- Concurrencia.
- Transacciones.
- Idempotencia.
- Seguridad.
- Performance.
- Resiliencia.
- Angular.
- RxJS.
- Manejo de estado.

- Testing.
- Observabilidad.
- Capacidad para justificar decisiones técnicas.

No se espera una arquitectura excesivamente compleja.

Se valorará más una solución simple, robusta y bien razonada que una solución sobrearquitecturada.

---

2. Escenario

Una empresa necesita desarrollar una plataforma para administrar pedidos.

Los clientes pueden:

- Consultar productos.
- Consultar inventario.
- Crear pedidos.
- Consultar sus pedidos.
- Cancelar pedidos.

El sistema también debe permitir que los administradores:

- Gestionen productos.
- Modifiquen inventario.
- Consulten pedidos.
- Cambien el estado de los pedidos.

La arquitectura inicial será:



```
SQL             Cache          External
Server                         Service
```

El candidato puede modificar esta arquitectura si considera que existe una razón técnica para hacerlo.

---

3. Modelo de dominio

Product

```
Product
---------------------------------
Id
Sku
Name
Description
Price
Stock
IsActive
Version
CreatedAt
UpdatedAt
```

Order

```
Order
---------------------------------
Id
CustomerId
CreatedAt
Status
Total
```

OrderItem

```
OrderItem
---------------------------------
Id
OrderId
ProductId
Quantity
UnitPrice
Subtotal
```

Customer

```
Customer
--------------------------------
Id
Name
Email
IsActive
```

Estados de Order:

```
Pending
Confirmed
Cancelled
Completed
```

---

4. Regla principal de negocio

Cuando un cliente crea una orden:

```
Customer


Create Order

  Validate customer

  Validate products

  Validate stock

  Obtain current prices

  Calculate total

  Reserve/decrease stock

  Create order
```

El precio enviado por el frontend **nunca debe considerarse confiable**.

El backend debe utilizar el precio actual almacenado en el servidor.

---

5. Backend

Implementar una ASP.NET Core Web API.

La solución debe utilizar:

- Dependency Injection.
- async/await.
- Entity Framework Core.
- DTOs.
- Validación.
- Logging.
- Manejo centralizado de excepciones.
- HTTP status codes apropiados.

---

6. Product API

Implementar:

`GET /api/products`

Debe soportar:

```
page
pageSize
search
sortBy
sortDirection
```

Ejemplo:

`GET /api/products?page=1&pageSize=20&search=laptop&sortBy=price&sortDirection=asc`

Debe implementar paginación a nivel de base de datos.

No se acepta:

```
_context.Products.ToList()
```

seguido de pagination en memoria.

---

7. Product Detail

`GET /api/products/{id}`

Debe devolver:

```
{
  "id": 10,
  "sku": "LAP-001",
  "name": "Laptop",
  "price": 1200,
  "stock": 15,
```

```
  "isActive": true,
}
```

---

8. Create Product

`POST /api/products`

Request:

```
{
  "sku": "LAP-001",
  "name": "Laptop",
  "description": "Business laptop",
  "price": 1200,
  "stock": 15,
}
```

Validaciones mínimas:

- SKU requerido.
- SKU único.
- Name requerido.
- Price > 0.
- Stock $>= 0$.

Debe evitarse la posibilidad de crear dos productos con el mismo SKU bajo concurrencia.

---

9. Update Product

`PUT /api/products/{id}`

Debe considerar concurrencia.

Por ejemplo:

```
Product Version = 5

User A obtiene Version 5
User B obtiene Version 5

User A actualiza → Version 6

User B intenta actualizar usando Version 5
```

La segunda operación no debería sobrescribir silenciosamente el cambio de User A.

El candidato debe implementar o explicar una estrategia de:

`Optimistic Concurrency`

---

10. Order API

Implementar:

`POST /api/orders`

Request:

```
{
    "customerId": 100,
    "items": [
        {
            "productId": 1,
            "quantity": 2
        },
        {
            "productId": 5,
            "quantity": 1
        }
    ]
}
```

El backend debe:

1. Validar customer.
2. Validar productos.
3. Validar que estén activos.
4. Obtener precios actuales.
5. Validar stock.
6. Calcular subtotales.
7. Calcular total.
8. Crear Order.
9. Crear OrderItems.
10. Actualizar inventario.
11. Mantener consistencia transaccional.

---

11. Concurrencia de inventario

Este es uno de los puntos principales de la prueba.

Supongamos:

```
Product Stock = 10
```

Llegan simultáneamente:

```
Request A → Quantity = 7

Request B → Quantity = 6
```

El sistema **no debe permitir vender 13 unidades**.

El candidato debe implementar una solución o explicar claramente cómo garantizaría:

`Stock >= Quantity`

de forma segura bajo concurrencia.

Se espera que considere alguna combinación de:

- Transactions.
- Optimistic concurrency.
- Pessimistic locking.
- Atomic update.
- RowVersion.
- Isolation levels.

No es necesario utilizar una técnica específica.

Se evaluará la **justificación técnica**.

---

12. Idempotencia

El cliente podría enviar dos veces:

`POST /api/orders`

debido a:

- Timeout.
- Retry.
- Problema de red.
- Doble click.
- Reconexión.

El sistema no debería crear dos órdenes accidentalmente.

El candidato debe implementar una estrategia de idempotencia.

Por ejemplo:

```
Idempotency-Key: 9f8c7d6a...
```

El diseño debe explicar:

- Dónde se almacena.

- Qué se considera una request duplicada.
- Qué ocurre si la primera request todavía está procesándose.
- Qué ocurre si la primera request falla.
- Cuánto tiempo debería conservarse la clave.

---

13. Transaction Boundary

El candidato debe definir claramente qué operaciones pertenecen a la misma transacción.

Por ejemplo:

```
Create Order
```

```
  Create Order
  Create OrderItems
  Update Stock
```

Si alguna operación falla:

```
Rollback
```

Debe evitarse una situación como:

```
Order created
Stock updated
OrderItems failed
```

sin una estrategia clara de recuperación.

---

14. External Service

Supongamos que después de crear la orden debemos llamar a:

`POST https://external-notification-service/api/notifications`

para enviar una notificación.

El servicio externo puede:

- Responder lentamente.
- Devolver HTTP 500.
- Tener timeout.
- Estar temporalmente caído.

El candidato debe explicar si esa llamada debería ejecutarse:

```
Dentro de la transacción
```

o:

`Fuera de la transacción`

y cómo garantizaría confiabilidad.

Se valorarán conceptos como:

- Background processing.
- Queue.
- Outbox pattern.
- Retry.
- Exponential backoff.
- Circuit breaker.
- Idempotency.

No es obligatorio implementar un message broker real.

---

15. Consultar órdenes

Implementar:

`GET /api/orders`

Parámetros:

```
page
pageSize
status
from
to
search
```

Debe soportar pagination.

La respuesta:

```
{
    "items": [],
    "page": 1,
    "pageSize": 20,
    "totalItems": 1200,
    "totalPages": 60
}
```

---

16. Order Detail

`GET /api/orders/{id}`

Debe devolver:

```
{
  "id": 10001,
  "customerId": 100,
  "customerName": "John Smith",
  "createdAt": "2026-09-04T10:30:00Z",
  "status": "Confirmed",
  "total": 1450,
  "items":
    [
      {
        "productId": 1,
        "productName": "Laptop",
        "quantity": 1,
        "unitPrice": 1200,
        "subtotal": 1200,
      },
      {
        "productId": 5,
        "productName": "Keyboard",
        "quantity": 1,
        "unitPrice": 250,
        "subtotal": 250,
      },
    ],
}
```

---

17. Cancel Order

Implementar:

`POST / api / orders / { id } / cancel;`

Reglas:

- Una orden Completed no puede cancelarse.
- Una orden Cancelled no puede cancelarse nuevamente.
- Al cancelar, el inventario debe restaurarse.
- La operación debe ser consistente.

El candidato debe considerar concurrencia.

---

18. Security

Implementar autenticación/autorización de forma simplificada o mockeada.

Deben existir al menos dos roles:

```
Customer;
Admin;
```

Reglas:

```
Customer
       Consultar productos
       Crear órdenes
       Consultar sus órdenes
       Cancelar sus órdenes

Admin
       Gestionar productos
       Gestionar inventario
       Consultar todas las órdenes
```

El backend debe reforzar las reglas.

No es suficiente ocultar funcionalidades en Angular.

---

19. API Error Handling

Utilizar un mecanismo consistente para errores.

Ejemplo:

```
{
  "type": "https://example.com/errors/insufficient-stock",
  "title": "Insufficient stock",
  "status": 409,
  "detail": "Product LAP-001 does not have enough stock.",
  "traceId": "abc-123",
}
```

Debe evitarse devolver:

```
{
    "stackTrace": "...",
    "connectionString": "...",
    "internalException": "..."
}
```

---

20. SQL / EF Core

La implementación debe evitar problemas comunes como:

N+1 queries No realizar:

```
Get orders
    ↓
foreach order
    ↓
query customer
```

Over-fetching No traer columnas/relaciones innecesarias.

Pagination en memoria No utilizar:

```
ToList()
Skip()
Take()
```

cuando el dataset es grande.

Debe aplicarse:

```
Skip()
Take()
ToListAsync()
```

sobre `IQueryable` cuando corresponda.

---

21. Performance

El candidato debe identificar posibles puntos de performance.

Por ejemplo:

`GET /api/orders`

puede devolver millones de registros.

Debe explicar cómo evitaría:

- Queries excesivamente grandes.
- N+1.
- Payloads enormes.
- Falta de índices.
- Tracking innecesario en queries read-only.
- Consultas repetidas.
- Problemas de paginación.

---

22. Angular

Crear una aplicación Angular para consumir la API.

Debe contener:

```
Dashboard
Products
Orders
Create Order
Order Detail
```

---

23. Product Screen

Implementar:

```
------------------------------------------------------
Products
------------------------------------------------------

Search: [_________________________]

Sort: [Price ] [ASC ]

------------------------------------------------------
SKU         Product        Price        Stock
------------------------------------------------------
LAP-001     Laptop         $1200         15
KEY-001     Keyboard       $120          50
MOU-001     Mouse          $35          100
------------------------------------------------------

                    < 1 2 3 4 5 >
```

Debe soportar:

- Search.
- Sorting.
- Pagination.
- Loading.
- Error.
- Empty state.

---

24. Create Order

Crear:

```
------------------------------------------------------
Create Order
------------------------------------------------------

Customer

[ John Smith                              ]

Products

[ Laptop   ]      Quantity [ - 2 + ]

[ Keyboard   ]    Quantity [ - 1 + ]

------------------------------------------------------

Subtotal                          $1440
------------------------------------------------------
Total                             $1440

                          [Create Order]
------------------------------------------------------
```

Debe permitir:

- Agregar productos.
- Eliminar productos.
- Modificar cantidades.
- Calcular subtotales.
- Mostrar total.

---

25. RxJS

La búsqueda debe implementar:

```
     valueChanges
          ↓
    debounceTime
          ↓
  distinctUntilChanged
          ↓
       switchMap
          ↓
        HTTP
```

Debe evitar:

```
subscribe(() => {
   http.get(...).subscribe(...)
})
```

cuando exista una alternativa RxJS más apropiada.

---

26. Manejo de estado

El candidato debe manejar apropiadamente estados como:

```
Loading
Success
Empty
Error
Saving
```

No se requiere utilizar una librería externa de state management.

Sin embargo, si el candidato utiliza una, debe justificarla.

---

27. HTTP Interceptor

Implementar un interceptor para manejar aspectos transversales.

Por ejemplo:

```
Authorization
Trace ID
Error handling
Loading indicator
```

No es necesario implementar todos.

---

28. Performance Angular

Considerar:

- OnPush.
- trackBy / track.
- Lazy loading.
- Evitar subscriptions innecesarias.
- Evitar rendering innecesario.
- Pagination.
- Componentes reutilizables.

No es necesario implementar todas las técnicas.

Se evaluará si el candidato sabe **cuándo utilizarlas y por qué**.

---

29. Testing

Implementar tests para la lógica crítica.

Backend

Mínimo:

```
CreateOrder_WithValidRequest_CreatesOrder

CreateOrder_WithInsufficientStock_ReturnsConflict

CreateOrder_WithInactiveProduct_ReturnsBadRequest

CreateOrder_IsIdempotent

CancelOrder_RestoresStock
```

Angular

Mínimo:

```
OrderForm_ShouldRequireCustomer;

OrderForm_ShouldRejectInvalidQuantity;

ProductSearch_ShouldDebounceRequests;
```

---

30. Observabilidad

Agregar logging suficiente para diagnosticar problemas.

Por ejemplo:

```
OrderCreated
OrderCreationFailed
OrderCancelled
InventoryUpdated
ExternalNotificationFailed
```

Los logs deberían incluir información útil como:

`OrderId`
`CustomerId`
`TraceId`

sin registrar información sensible innecesaria.

---

31. Health Check

Implementar:

`GET /health`

Debe permitir determinar si la aplicación está funcionando.

Opcionalmente:

`Database`
`External dependencies`

pueden tener health checks independientes.

---

32. Docker

Opcional pero recomendado.

Idealmente:

`docker-compose`

`API`
`Angular`
`SQL Server`

No es necesario configurar Kubernetes.

---

33. Documentación

El proyecto debe incluir:

`README.md`

El README debe explicar:

Setup

Cómo ejecutar el proyecto.

Architecture

Explicar la arquitectura.

Database

Cómo crear/migrar la base de datos.

API

Endpoints principales.

Decisions

Explicar decisiones importantes.

Concurrency

Explicar cómo se protege el inventario.

Idempotency

Explicar cómo se evita crear órdenes duplicadas.

External Services

Explicar cómo manejaría fallos del servicio externo.

Improvements

Indicar qué cambiaría si el sistema tuviera que soportar:

```
10x traffic
100x traffic
```

---

34. Parte arquitectónica obligatoria

Además del código, responder en el README:

Escenario A

La aplicación actualmente recibe:

`1,000 requests/minute`

En seis meses se esperan:

`50,000 requests/minute`

¿Qué cambiarías?

Escenario B

SQL Server comienza a utilizar:

90% CPU

¿Qué investigarías primero?

Escenario C

Los usuarios reportan:

> "A veces se crean dos órdenes."

¿Cómo investigarías el problema?

Escenario D

El servicio externo de notificaciones está caído durante 30 minutos.

¿Las órdenes deberían dejar de crearse?

Explica tu decisión.

---

35. Restricciones

No es obligatorio implementar:

- Microservices.
- Kubernetes.
- Kafka.
- RabbitMQ.
- Event Sourcing.
- CQRS completo.
- Redis.
- GraphQL.

El candidato puede utilizar estas tecnologías si considera que aportan valor, pero debe justificarlo.

Agregar tecnología no suma puntos por sí mismo.

---

36. Entregables

Entregar:

```
/project
    /backend
    /frontend
    README.md
```

Opcional:

```
docker-compose.yml
```

No incluir:

```
node_modules
bin
obj
```

---

37. Criterios de evaluación

Área	Puntos
C# / .NET	10
ASP.NET Core / API	10
EF Core / SQL	10
Concurrency / Transactions	10
Architecture	10
Angular	10
RxJS / State Management	8
Security	7
Performance	7
Testing	7
Error Handling / Resilience	6
Documentation / Technical Decisions	5
Total	**100**

---

38. Bonus

Hasta 10 puntos adicionales.

+2 Outbox pattern correctamente implementado.

+2 Optimistic concurrency correctamente implementada.

+1 Docker completo.

+1 CI/CD.

+1 Integration tests.

+1 Distributed tracing / correlation IDs.

+1 Caching correctamente justificado.

+1 Angular lazy loading + performance optimization.

+1 Excellent documentation / architecture diagram.

---

39. Rúbrica de Seniority

90–100

Senior fuerte El candidato:

- Diseña pensando en producción.
- Entiende concurrencia.
- Comprende transacciones.
- Implementa APIs limpias.
- Tiene buen dominio de Angular/RxJS.
- Piensa en seguridad.
- Entiende performance.
- Maneja fallos.
- Puede explicar trade-offs.

80–89

Senior Cumple claramente con las expectativas.

Puede tener algunas áreas menores de mejora.

70–79

Mid/Senior Tiene buena capacidad de implementación, pero presenta debilidades en:

- Arquitectura.
- Concurrencia.

- Resiliencia.
- Performance.

Debe validarse en entrevista.

60–69

Mid-Level fuerte Puede implementar funcionalidades complejas, pero todavía requiere supervisión en decisiones arquitectónicas.

<60

No alcanza Senior

---

40. Red Flags

Estas señales deben considerarse seriamente.

Red Flag 1

El candidato confía en Angular para proteger operaciones.

```
"El botón no aparece para Customer"
```

No es suficiente.

Red Flag 2

No considera concurrencia de inventario.

Si simplemente hace:

```
product.Stock -= quantity;
```

sin analizar concurrencia, es una debilidad importante.

Red Flag 3

Utiliza retry sin idempotencia.

Especialmente para:

```
Create Order
Payment
Inventory
```

Red Flag 4

Mantiene una llamada HTTP externa dentro de una transacción SQL prolongada.

Debe explicar claramente el trade-off.

Red Flag 5

Utiliza microservices para resolver todo.

Ejemplo:

```
Product Service
Order Service
Customer Service
Inventory Service
Notification Service
Payment Service
```

sin explicar por qué.

Red Flag 6

No utiliza pagination.

Especialmente en:

```
Orders
Products
Customers
```

Red Flag 7

Tiene N+1 queries.

Red Flag 8

Usa `any` extensivamente en Angular.

Red Flag 9

Utiliza subscriptions anidadas constantemente.

Red Flag 10

No puede explicar sus decisiones.

Esta es probablemente la red flag más importante.

---

41. Preguntas de revisión durante la entrevista

Después de recibir el código, realizar una sesión de aproximadamente 45–60 minutos.

Pregunta 1

> ¿Cuál consideras que es la parte más importante de tu solución?

Busca entender prioridades.

Pregunta 2

> ¿Qué sucede si dos usuarios compran simultáneamente el último producto disponible?

Debe poder explicar exactamente qué ocurre.

Pregunta 3

> ¿Qué sucede si el frontend recibe timeout después de que el backend creó la orden?

Busca idempotencia.

Pregunta 4

> ¿Qué sucede si SQL Server confirma la orden pero el servicio de notificaciones está caído?

Busca capacidad de escal resiliencia y separación de responsabilidades.

Pregunta 5

> ¿Qué parte de esta solución cambiarías si tuvieras que soportar 100 veces más tráfico?

Busca capacidad de escalamiento.

Pregunta 6

> ¿Dónde colocarías caching?

Más importante que la respuesta es la justificación.

Pregunta 7

> ¿Qué query de tu solución podría convertirse en un problema con 10 millones de registros?

Busca pensamiento de performance.

Pregunta 8

> ¿Qué harías si una actualización de producto fuera sobrescrita accidentalmente por otro usuario?

Busca optimistic concurrency.

Pregunta 9

> ¿Por qué utilizaste `switchMap`?

Debe poder explicar el comportamiento y no solamente reconocer el patrón.

Pregunta 10

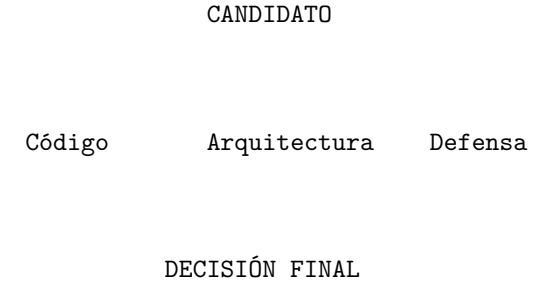
> Si mañana otro equipo consume tu API, ¿qué cambiarías antes de publicarla?

Busca madurez en API design.

---

42. Decisión final

La evaluación debería considerar tres dimensiones:



Un candidato Senior debe demostrar las tres.

Un código perfecto técnicamente pero que el candidato no puede explicar debe generar dudas.

Del mismo modo, una excelente conversación arquitectónica no compensa completamente una implementación deficiente.

La combinación de:

```
            Código
              +
         Arquitectura
              +
    Experiencia práctica
              +
    Capacidad de diagnóstico
              +
     Comunicación técnica
```

es la que debería determinar la decisión final.

Una diferencia importante respecto al Mid Para **Mid** yo pondría el peso en *"¿puede construir correctamente una feature?"*.

Para **Senior**, esta prueba pregunta además:

> **"¿Puede construirla de forma que sobreviva a concurrencia, fallos, crecimiento y cambios futuros?"**

Y hay cuatro puntos que yo **no negociaría para un Senior Full Stack**: **concurrencia de inventario, idempotencia, diseño de API y capacidad de explicar trade-offs**. Si falla fuertemente en esos cuatro, aunque el CRUD funcione muy bien, no lo consideraría Senior.